



Università Politecnica delle Marche

Facoltà di Ingegneria

Dipartimento di Ingegneria dell'Informazione

Corso di Laurea Magistrale in
Ingegneria Informatica e dell'Automazione

Sviluppo di una Zero Trust Architecture per la gestione di un impianto di energia nucleare

Docente

Prof. Luca Spalazzi

Componenti del gruppo

Fabio Tempera
Luca Marcianesi
Gabriele Vianello
Gianluca Baldelli

Anno Accademico 2025-2026

Indice

1	Introduzione	5
1.1	Il progetto	5
1.2	Obiettivi	6
1.3	Struttura dell'elaborato	6
2	Fondamenti teorici	7
2.1	Il modello Zero Trust	7
2.1.1	Principi fondamentali secondo NIST 800-207	7
2.1.2	Policy Enforcement Point e Policy Decision Point	8
2.2	Modelli formali di controllo degli accessi	8
2.2.1	Controllo degli accessi basato su ruoli	8
2.2.2	Controllo degli accessi basato su attributi	9
2.2.3	Il modello Bell-LaPadula	9
2.2.4	Algoritmo di fiducia	10
2.3	Principi di progettazione sicura	11
2.3.1	Economy of mechanism	11
2.3.2	Fail-safe defaults	11
2.3.3	Complete mediation	12
2.3.4	Separation of privilege	12
2.3.5	Least privilege	12
2.3.6	Open design	13
2.3.7	Psychological acceptability	13
2.4	Crittografia applicata e TLS	13
2.4.1	Transport Layer Security	13
2.4.2	Mutual TLS	14
2.4.3	JA3 fingerprinting	14
2.5	Componenti tecnologici	15
2.5.1	nftables	15
2.5.2	Snort	15
2.5.3	Envoy Proxy	16
2.5.4	Open Policy Agent e Rego	16
2.5.5	Splunk	17
2.5.6	MongoDB	17
2.5.7	Docker e Docker Compose	17
2.5.8	Go	18
2.5.9	Protobuf e gRPC	18
3	Architettura del sistema	20
3.1	Struttura della repository	20

3.2	La segmentazione di rete	21
3.3	Il firewall e il punto di ingresso	22
3.4	Envoy Proxy come Policy Enforcement Point	22
3.5	Il Security Orchestrator come Policy Administrator	23
3.5.1	Alert provenienti dagli Snort	24
3.6	OPA come Policy Decision Point	24
3.7	Il Business Logic service	24
3.8	Identity Access Management service	25
3.9	I database	25
3.10	Il sistema di rilevamento delle intrusioni	26
3.10.1	Meccanismo di Inoltro degli Alert	26
3.11	Il frontend	26
3.12	Il seeder	27
3.13	Il flusso di autorizzazione end-to-end	27
4	Implementazione dei componenti	28
4.1	Il firewall di rete con nftables	28
4.2	Il sistema di rilevamento intrusioni con Snort	30
4.3	Envoy Proxy e la pipeline TLS	31
4.4	Il Security Orchestrator	33
4.5	Open Policy Agent e la policy in Rego	34
4.6	Il servizio Identity	35
4.7	Il servizio Business Logic	37
4.8	I database MongoDB	38
4.9	Il seeder del Business DB	38
4.10	Il forwarder Splunk e l'osservabilità centralizzata	39
4.11	La suite di test e validazione automatizzata	39
5	Il modello Graphagate: Temporal Graph Network per la Zero Trust Architecture	41
5.1	Introduzione	41
5.2	Il modello base: Temporal Graph Network (v3)	41
5.2.1	Architettura del sistema	41
5.2.2	Fase di addestramento e calibrazione	44
5.2.3	Tipologie di anomalie rilevate in produzione	45
5.2.4	Valutazione sperimentale e confronto con le baseline	45
5.3	L'evoluzione: il nodo <i>Configuration</i> (schema v4)	46
5.3.1	Motivazione	46
5.3.2	Architettura del nodo <i>Configuration</i>	47
5.3.3	Risultati: il contributo del nodo config	48
5.3.4	Esperimento: granularità dell'identità del dispositivo (nodo <code>dev:guest</code>)	50
5.4	Deployment e specifiche hardware/software	51
5.4.1	Piattaforma HW/SW (addestramento e inferenza)	52
5.4.2	Serving e latenza di inferenza	52
5.4.3	Parametri di inferenza forniti dal Security Orchestrator	52

5.5	Limiti del modello	54
5.5.1	Assenza di scalabilità orizzontale	54
5.5.2	La sfida del lateral movement	55
5.5.3	Anti-poisoning gate nel live stream	55
5.6	Quadro riepilogativo: confronto di tutti i modelli	56
6	Validazione e Test delle API (Postman)	57
6.1	Ambiente di Test e Premessa Metodologica	57
6.2	Analisi dei Risultati e Scenari di Carico	57
6.2.1	PC portatile	58
6.2.2	PC fisso	58

Elenco delle figure

2.1	Tabella BellLapadula	10
3.1	Immagine architettura	20
5.1	Schema dei layer del modello: i tre hop di <i>GraphAttentionEmbedding</i> e le due teste (<i>Feature</i> e <i>Structural</i>) del <i>Dual Head Scorer</i>	43
5.2	Catena causale dello schema v4, con il nodo <i>config</i> inserito tra <i>source</i> e <i>device</i> e il <i>binding config</i> → <i>user</i> ; gli archi tratteggiati sono <i>binding</i> a messaggio-zero, il solo <i>access user</i> → <i>resource</i> porta il messaggio a 7 dimensioni.	47
6.1	Grafico Peak 30 UV	58
6.2	Grafico Peak 100 UV	59

1 Introduzione

La sicurezza informatica nei sistemi distribuiti è un campo in continua evoluzione, spinto dalla crescente complessità delle infrastrutture moderne e dall'inadeguatezza dei modelli di protezione tradizionali. Il paradigma del perimetro difensivo, in cui tutto ciò che si trova all'interno della rete aziendale è considerato affidabile per definizione, ha mostrato i propri limiti in modo sempre più evidente negli ultimi anni. Violazioni sofisticate, movimenti laterali non rilevati e compromissioni di credenziali legittime hanno reso chiaro che la fiducia implicita nella rete interna è una premessa insostenibile.

Da questa consapevolezza nasce il modello Zero Trust, la cui idea fondante può essere riassunta in un unico principio: non fidarsi mai, verificare sempre. In un'architettura Zero Trust, nessuna richiesta di accesso viene considerata sicura a priori, indipendentemente dalla provenienza, dall'identità dichiarata o dalla rete di appartenenza. Ogni accesso deve essere autorizzato esplicitamente, valutando in modo continuo il contesto dell'identità richiedente, del dispositivo utilizzato, della risorsa a cui si vuole accedere e del livello di rischio associato alla transazione.

1.1 Il progetto

Questo elaborato descrive la progettazione e l'implementazione di un'architettura Zero Trust basata su microservizi, sviluppata nell'ambito del corso di Advanced Cybersecurity for IT. Lo scenario scelto come dominio applicativo è quello di una centrale nucleare, un contesto volutamente estremo per quanto riguarda i requisiti di sicurezza: ruoli operativi con responsabilità nettamente distinte, informazioni classificate a vari livelli di sensibilità, accesso fisico e digitale strettamente correlati e conseguenze potenzialmente catastrofiche in caso di compromissione.

L'architettura è distribuita su sei container Docker — Envoy Proxy, Security Orchestrator, Open Policy Agent, Business Logic, Security Database e Business Database — e si conforma ai principi definiti dalla pubblicazione NIST Special Publication 800-207, il riferimento tecnico consolidato per le implementazioni Zero Trust. Il fulcro del progetto è la separazione netta tra il Policy Enforcement Point (PEP), responsabile dell'intercettazione del traffico e dell'applicazione delle decisioni, e il Policy Decision Point (PDP), che valuta le policy e produce il verdetto di accesso. Questa separazione garantisce che nessun componente concentri in sé sia la capacità di applicare le regole sia quella di definirle, riducendo la superficie di attacco e aumentando la chiarezza architetturale.

Il sistema implementa un controllo degli accessi adattivo e basato sul rischio, che va ben oltre la semplice validazione delle credenziali. Ogni richiesta viene analizzata tenendo conto dell'impronta TLS del client tramite JA3 fingerprinting, della presenza di un certificato client valido mediante mutual TLS, del trust score dell'identità richiedente calcolato a partire dall'analisi comportamentale e del livello di classificazione della risorsa richiesta.

Il risultato è un sistema capace di prendere decisioni di accesso granulari, contestuali e dinamiche.

1.2 Obiettivi

Gli obiettivi del progetto sono articolati su più livelli. Sul piano architetturale, l'obiettivo è dimostrare come i principi Zero Trust possano essere applicati concretamente in un'infrastruttura containerizzata, con una segmentazione di rete rigorosa e una catena di autorizzazione verificabile end-to-end. Sul piano della sicurezza, si vuole mostrare come sia possibile implementare un controllo degli accessi che tenga conto simultaneamente di identità, dispositivo, rete, comportamento e classificazione della risorsa, superando i limiti dei modelli basati esclusivamente su ruoli o credenziali statiche.

Sul piano dell'osservabilità, un obiettivo esplicito è garantire la tracciabilità completa di ogni richiesta attraverso tutti i componenti del sistema, mediante la propagazione di un identificatore di correlazione univoco (`X-Request-ID`) e la centralizzazione dei log su Splunk. Tale integrazione consente sia di ricostruire il percorso di ogni transazione, sia di rilevare scenari di attacco complessi attraverso interrogazioni e allarmi configurati sulla piattaforma di monitoraggio.

Sul piano della rilevazione delle intrusioni, il progetto integra un firewall di rete basato su `nftables`, posizionato come primo punto di accesso all'infrastruttura, e un sistema di rilevamento delle intrusioni basato su `Snort`, configurato per individuare pattern di traffico anomalo come la scansione delle porte. L'insieme di questi livelli di difesa concorre a realizzare una protezione in profondità coerente con i principi Zero Trust.

1.3 Struttura dell'elaborato

L'elaborato è organizzato in modo da accompagnare il lettore dalla teoria alla pratica in modo progressivo. Il Capitolo 2 fornisce il quadro teorico completo, trattando i fondamenti della crittografia applicata, i modelli formali di controllo degli accessi, i principi di progettazione sicura e le tecnologie specifiche impiegate nel progetto. Il Capitolo 3 descrive l'architettura del sistema nei suoi componenti, flussi e scelte progettuali. Il Capitolo 4 entra nel dettaglio implementativo di ciascun modulo. Il Capitolo 5 presenta i test eseguiti, gli scenari di attacco simulati e i risultati osservati. Il Capitolo 6 conclude con una valutazione critica del lavoro svolto e le possibili evoluzioni future.

2 Fondamenti teorici

Prima di addentrarsi nei dettagli implementativi dell'architettura realizzata, è necessario costruire un solido quadro teorico che permetta di comprendere le scelte progettuali con piena consapevolezza del loro significato. Questo capitolo tratta i modelli formali di sicurezza, i principi di progettazione sicura, le tecnologie crittografiche alla base delle comunicazioni protette e gli strumenti specifici impiegati nel progetto.

2.1 Il modello Zero Trust

Il termine Zero Trust fu coniato da John Kindervag nel 2010, durante il suo lavoro presso Forrester Research, ma le sue radici concettuali affondano in una critica più antica al modello perimetrale. L'idea tradizionale di sicurezza informatica era quella del castello con il fossato: una rete interna considerata sicura, protetta da un perimetro difensivo verso l'esterno. Tutto ciò che si trovava all'interno del perimetro godeva di un livello implicito di fiducia. Questa assunzione si è rivelata pericolosa: una volta violato il perimetro, un attaccante poteva muoversi lateralmente attraverso la rete con scarsa resistenza.

Il modello Zero Trust rovescia questa logica in modo radicale. Nessuna entità, né utente, né dispositivo, né servizio, è considerata affidabile per default, indipendentemente dalla sua posizione nella rete. Ogni accesso deve essere esplicitamente autenticato, autorizzato e continuamente validato. La rete stessa smette di essere un elemento di fiducia e diventa semplicemente un mezzo di trasporto.

Il riferimento normativo più autorevole per l'implementazione pratica del modello è la pubblicazione NIST SP 800-207, *Zero Trust Architecture*, pubblicata nel 2020. Questo documento definisce i principi fondamentali del modello e identifica i componenti logici di un'architettura Zero Trust.

2.1.1 Principi fondamentali secondo NIST 800-207

La pubblicazione NIST SP 800-207 articola il modello Zero Trust attorno a sette principi fondamentali:

- tutte le risorse sono considerate non affidabili, indipendentemente dalla loro posizione nella rete;
- tutte le comunicazioni devono essere protette, anche quelle interne alla rete aziendale;
- l'accesso alle risorse deve essere concesso su base per-sessione e con il minimo privilegio necessario;

- l'accesso è determinato da policy dinamiche che tengono conto dell'identità, del dispositivo, della rete e di altri attributi contestuali;
- l'integrità e la sicurezza di tutti i dispositivi connessi devono essere monitorate e verificate;
- l'autenticazione e l'autorizzazione devono essere eseguite in modo dinamico e rigoroso prima di consentire l'accesso;
- l'organizzazione deve raccogliere quante più informazioni possibile sullo stato delle risorse e delle comunicazioni, e usarle per migliorare continuamente la postura di sicurezza.

2.1.2 Policy Enforcement Point e Policy Decision Point

L'architettura Zero Trust secondo NIST si articola attorno a due componenti logici principali. Il Policy Enforcement Point (PEP) è il punto in cui il traffico viene intercettato e le decisioni di accesso vengono applicate. Il PEP non decide autonomamente: riceve un verdetto da un'entità separata e lo esegue, consentendo o bloccando la richiesta. Il Policy Decision Point (PDP) è invece il componente che valuta le policy e produce il verdetto. Nel modello NIST, il PDP si compone a sua volta di un Policy Engine, che applica le regole di accesso, e di un Policy Administrator, che traduce il verdetto in istruzioni operative per il PEP.

La separazione tra PEP e PDP è un requisito architetturale fondamentale, non una mera convenzione. Essa garantisce che la logica di applicazione delle regole e la logica di definizione delle regole siano mantenute distinte, riducendo la complessità di ciascun componente e limitando il raggio di impatto di una potenziale compromissione.

Nel progetto descritto in questo elaborato, il PEP è implementato da Envoy Proxy, mentre il PDP è realizzato dalla combinazione del Security Orchestrator, del modello di AI e dal motore di policy OPA.

2.2 Modelli formali di controllo degli accessi

La teoria del controllo degli accessi ha radici che risalgono agli anni Settanta e ha prodotto modelli formali che ancora oggi informano la progettazione dei sistemi di sicurezza moderni. Comprendere questi modelli è essenziale per valutare le scelte architetturali di qualsiasi sistema che gestisca risorse con livelli di sensibilità differenziati.

2.2.1 Controllo degli accessi basato su ruoli

Il Role-Based Access Control (RBAC), standardizzato dal NIST nella pubblicazione ANSI/INCITS 359-2004, è un modello di controllo degli accessi in cui i permessi non sono assegnati direttamente agli utenti ma ai ruoli, e gli utenti acquisiscono i permessi in virtù

dell'appartenenza a uno o più ruoli. Questo approccio semplifica notevolmente la gestione degli accessi in sistemi con molti utenti e molte risorse.

Nel progetto il modello Role-Based Access Control costituisce il livello di base per l'astrazione e la gestione delle identità all'interno del sistema di autorizzazione. A differenza delle implementazioni RBAC tradizionali, dove il ruolo determina direttamente e staticamente i permessi di accesso a specifiche risorse, in questa architettura il ruolo funge da livello di disaccoppiamento organizzativo. Le identità degli utenti vengono veicolate tramite token JWT dai quali viene estratto un attributo di business (ad esempio: `guest`, `operator`, `manager`, `admin`). Il motore di policy utilizza una mappa associativa per convertire questi ruoli organizzativi in etichette di sicurezza.

Invece di assegnare individualmente complessi set di parametri ad ogni singolo dipendente, l'appartenenza a un ruolo popola dinamicamente gli attributi strutturali necessari ai modelli successivi.

2.2.2 Controllo degli accessi basato su attributi

Il modello Attribute-Based Access Control (ABAC) estende RBAC introducendo la valutazione di attributi arbitrari relativi al soggetto, alla risorsa, all'ambiente e all'azione richiesta. Le decisioni di accesso non dipendono solo dal ruolo dell'utente, ma da una combinazione di attributi che può includere l'ora della richiesta, il tipo di dispositivo, il livello di rischio corrente e qualsiasi altra informazione contestuale rilevante.

Nell'implementazione realizzata, l'approccio ABAC permette di superare i limiti di un'autorizzazione puramente statica valutando il contesto di ogni singola richiesta in tempo reale. Agli attributi dell'oggetto (come il percorso specifico dell'endpoint e il metodo HTTP) vengono associati metadati quantitativi intrinseci di rischio.

L'elemento architetturale chiave introdotto in questa fase è la valutazione degli attributi ambientali, rappresentati specificamente da uno *score di rischio* continuo. Attraverso questo scambio di attributi, l'ABAC realizza concretamente il principio della valutazione continua del rischio.

2.2.3 Il modello Bell-LaPadula

Il modello Bell-LaPadula, proposto da David Elliott Bell e Leonard J. LaPadula nel 1973 per conto della United States Air Force, è il modello formale di riferimento per la confidenzialità delle informazioni. Il suo obiettivo è impedire che informazioni riservate fluiscano verso soggetti non autorizzati a riceverle.

Il modello introduce il concetto di livello di sicurezza, originariamente costituito da due dimensioni che formano una struttura a reticolo: una gerarchia totalmente ordinata e un insieme non gerarchico di compartimenti o categorie. Nel progetto realizzato, i livelli gerarchici adottati sono cinque: PUBLIC, INTERNAL, CONFIDENTIAL, SECRET e TOP_SECRET, in ordine crescente di sensibilità. A questi si affiancano specifiche categorie (es. *nuclear*, *security*). Un accesso è consentito solo se, oltre a soddisfare i vincoli

di livello, il soggetto possiede un insieme di categorie che include (superset) tutte quelle richieste dall'oggetto.

Il modello implementato si fonda sulle due proprietà fondamentali del Mandatory Access Control (MAC). La *Simple Security Property* (o proprietà *no read-up*) stabilisce che un soggetto può leggere un oggetto solo se il suo livello di autorizzazione domina quello dell'oggetto. La *Star Property* (o proprietà *no write-down*) stabilisce che un soggetto non può scrivere su un oggetto con livello di classificazione inferiore al proprio, impedendo il declassamento illecito delle informazioni.

Esempio di sanificazione

Tuttavia, l'implementazione in OPA estende questo concetto introducendo la figura del *Trusted Subject*: viene definita una deroga controllata alla *Star Property* per consentire a un utente **admin** (TOP_SECRET) di scrivere su una specifica rotta sanitizzata di livello inferiore (SECRET), permettendo così una de-classificazione sicura delle informazioni.

A	B	C	D	E	F	G	H
Rotta Base	Metodo	Tipo Azione	Classif. Risorsa	Guest(Clr: 0 / Cat: nessuna)	Operator(Clr: 1 / Cat: hr, ops)	Manager(Clr: 2 / Cat: hr, ops, fin)	Admin(Clr: 4 / Cat: tutte)
/personnel	GET	Lettura	INTERNAL (1)	✗ No Cat	CONCESSO	CONCESSO	CONCESSO
	POST	Scrittura	INTERNAL (1)	✗ No Cat	CONCESSO	✗ BLOCCATO (+Prop)	✗ BLOCCATO (+Prop)
/documents	GET	Lettura	CONFIDENTIAL (2)	✗ No Cat	CONCESSO	CONCESSO	CONCESSO
	POST	Scrittura	CONFIDENTIAL (2)	✗ No Cat	✗ No Cat	CONCESSO	✗ BLOCCATO (+Prop)
	DELETE	Scrittura	CONFIDENTIAL (2)	✗ No Cat	✗ No Cat	CONCESSO	✗ BLOCCATO (+Prop)
/nuclear-materia	GET	Lettura	TOP_SECRET (4)	✗ No Cat	✗ No Cat	✗ No Cat	CONCESSO
	POST	Scrittura	TOP_SECRET (4)	✗ No Cat	✗ No Cat	✗ No Cat	CONCESSO (454)
	DELETE	Scrittura	TOP_SECRET (4)	✗ No Cat	✗ No Cat	✗ No Cat	CONCESSO (454)
/reactor-paramet	GET	Lettura	TOP_SECRET (4)	✗ No Cat	✗ No Cat	✗ No Cat	CONCESSO
	POST	Scrittura	TOP_SECRET (4)	✗ No Cat	✗ No Cat	✗ No Cat	CONCESSO (454)
/trusted-guard/	POST	Scrittura	SECRET (3)	✗ No Cat	✗ No Cat	✗ No Cat	CONCESSO (Eccozione)

Figura 2.1: Tabella BellLapadula

2.2.4 Algoritmo di fiducia

L'algoritmo di fiducia implementato nel policy.rego di Opa si distingue per la sua architettura *ibrida*, che fonde metodologie di **controllo rigido degli attributi** (tipiche dei sistemi Mandatory Access Control) con meccanismi di **controllo basato sul punteggio**.

Controllo degli Attributi: Modello Bell-LaPadula

La componente deterministica dell'algoritmo si basa su un'implementazione del modello di confidenzialità Bell-LaPadula (BLP).

Controllo basato sul punteggio: valutazione Dinamica del Rischio

A valle del controllo deterministico, l'algoritmo ibrido introduce una fase di computazione algoritmica basata sulla somma e sottrazione di attributi di rischio continuo. Questo garantisce che credenziali compromesse (ma formalmente valide nel modello BLP) non siano sufficienti per finalizzare un attacco.

Per ogni singola richiesta HTTP, l'algoritmo valuta:

- **Beneficio dell'Operazione (B):** Un indice quantitativo pre-assegnato al metodo e alla rotta (es. 0.9 per rotte critiche, 0.6 per risorse interne).

- **Soglia di Rischio Accettato (R_{acc}):** Il livello limite di anomalia tollerabile per la specifica transazione.
- **Punteggio AI di Minaccia (A):** Un attributo dinamico valutato in tempo reale che quantifica il livello di rischio della richiesta.

Il blocco di autorizzazione finale si risolve tramite una disuguaglianza algebrica che calcola il *Rischio Netto*:

$$B - A \geq R_{acc} \quad (2.1)$$

Questa disuguaglianza impone che un'operazione venga completata unicamente se il suo vantaggio contestuale eccede rigorosamente la soglia di rischio accettato configurata per lo specifico endpoint.

2.3 Principi di progettazione sicura

Parallelamente ai modelli formali, la letteratura di sicurezza informatica ha elaborato un insieme di principi di progettazione che guidano la costruzione di sistemi resistenti. Questi principi, enunciati originariamente da Jerome Saltzer e Michael Schroeder nel loro articolo seminale del 1975 *The Protection of Information in Computer Systems*, rimangono di piena attualità e informano ogni scelta architetturale significativa.

2.3.1 Economy of mechanism

Il principio di economy of mechanism afferma che i meccanismi di sicurezza devono essere progettati per essere il più semplici e piccoli possibile. La semplicità riduce la probabilità di errori implementativi, semplifica la verifica della correttezza e limita la superficie di attacco. Un meccanismo complesso è più difficile da capire, da testare e da mantenere nel tempo.

Nel progetto, questo principio si manifesta nella scelta di componenti con responsabilità ben definite e limitate. Ogni container ha un compito preciso: nftables si occupa esclusivamente del filtraggio del traffico a livello di rete, Envoy gestisce la terminazione TLS e la delega dell'autorizzazione, OPA valuta le policy in linguaggio Rego. Nessun componente è sovraccaricato di responsabilità che appartengono ad altri.

2.3.2 Fail-safe defaults

Il principio di fail-safe defaults stabilisce che, in assenza di una decisione di autorizzazione esplicita, il sistema deve negare l'accesso. Il comportamento predefinito deve essere il rifiuto, non il consenso. Un sistema che fallisce in modo sicuro limita i danni derivanti da errori di configurazione o da condizioni impreviste.

Questo principio è visibile in più punti dell'architettura. La policy OPA definisce `default allow = false`, garantendo che qualsiasi richiesta per cui non sia applicabile una regola

esplicita di consenso venga negata automaticamente. Analogamente, la catena di `iftables` configura la policy di default del chain `input` a `DROP`, in modo che solo il traffico esplicitamente consentito possa raggiungere il sistema.

2.3.3 Complete mediation

Il principio di complete mediation afferma che ogni accesso a ogni risorsa deve essere verificato, senza eccezioni e senza possibilità di aggirare il meccanismo di controllo. Non deve esistere alcun percorso che consenta di accedere a una risorsa senza passare attraverso il punto di autorizzazione.

L'architettura del progetto garantisce la complete mediation attraverso la topologia di rete. Il Business DB, che contiene i dati più sensibili dell'applicazione, è raggiungibile esclusivamente dal servizio Business Logic attraverso la rete isolata Back-Net. Non esiste alcun percorso diretto dall'esterno o dal Security Orchestrator verso la Business DB. Ogni richiesta al servizio applicativo deve necessariamente transitare per Envoy, che intercetta il traffico e delega l'autorizzazione prima di instradare la richiesta.

2.3.4 Separation of privilege

Il principio di separation of privilege afferma che un sistema non dovrebbe concedere l'accesso o eseguire un'operazione sulla base di un'unica condizione, ma richiedere la soddisfazione di più condizioni indipendenti. La separazione dei privilegi riduce il rischio che la compromissione di un singolo fattore sia sufficiente a violare la sicurezza del sistema.

Nel progetto, la decisione di accesso richiede la soddisfazione simultanea di più condizioni: l'utente deve avere un livello di clearance adeguato alla classificazione della risorsa, un trust score superiore alla soglia minima richiesta dalla zona, l'enrollment MFA dove richiesto, un dispositivo di tipo consentito e una provenienza di rete autorizzata. La compromissione di un solo fattore, ad esempio il furto di credenziali, non è sufficiente a ottenere l'accesso se il dispositivo utilizzato non corrisponde all'impronta JA3 attesa o se il trust score è al di sotto della soglia.

2.3.5 Least privilege

Il principio del minimo privilegio afferma che ogni entità del sistema, utente, processo o componente, deve operare con il minimo insieme di permessi necessario per svolgere le proprie funzioni. Questo limita i danni in caso di compromissione e riduce la superficie di attacco disponibile.

L'applicazione di questo principio è pervasiva nell'architettura. Il servizio Business Logic non si connette al database con un'unica utenza onnipotente, ma utilizza tre credenziali MongoDB distinte e role-scoped (`operator_client`, `manager_client`, `admin_client`), ciascuna abilitata alle sole collection pertinenti al rispettivo livello. In questo modo il principio del minimo privilegio è applicato non solo dalla policy di autorizzazione, ma anche dalle credenziali con cui si raggiunge effettivamente il dato, fornendo un secondo

strato di difesa indipendente. Ogni microservizio, analogamente, opera all'interno del segmento di rete strettamente necessario alle proprie funzioni.

2.3.6 Open design

Il principio di open design afferma che la sicurezza di un sistema non deve dipendere dalla segretezza del suo meccanismo di protezione, ma esclusivamente dalla segretezza delle chiavi o delle credenziali. Un sistema la cui sicurezza dipende dall'oscurità della propria implementazione è intrinsecamente fragile: se il meccanismo viene scoperto, la sicurezza crolla completamente.

Nel progetto, i meccanismi di protezione sono basati su standard aperti e ampiamente verificati: TLS 1.3 per la cifratura delle comunicazioni, X.509 per i certificati, SHA-256 per gli hash di integrità. La sicurezza risiede nella segretezza delle chiavi private, non nella segretezza degli algoritmi. Le policy OPA sono espresse in Rego, un linguaggio leggibile e auditabile, la cui correttezza può essere verificata in modo indipendente.

2.3.7 Psychological acceptability

Il principio di psychological acceptability afferma che i meccanismi di sicurezza devono essere progettati in modo da non creare ostacoli eccessivi agli utenti legittimi. Se i controlli di sicurezza sono troppo onerosi, gli utenti troveranno modi per aggirarli. La sicurezza deve essere praticabile, non solo teoricamente corretta.

In un'architettura Zero Trust, questo principio si traduce nella trasparenza dei controlli: l'autenticazione e l'autorizzazione avvengono in modo il più possibile automatico, senza richiedere azioni aggiuntive all'utente se non nei casi in cui il rischio lo giustifichi. L'autenticazione a più fattori è sempre richiesta al login: dopo la verifica di username e password, il sistema emette un codice OTP monouso inviato via email che l'utente deve confermare prima che venga rilasciato il token di sessione.

2.4 Crittografia applicata e TLS

Le comunicazioni sicure all'interno dell'architettura si fondano su protocolli crittografici standard. La comprensione dei meccanismi sottostanti è necessaria per valutare le garanzie di sicurezza offerte dal sistema.

2.4.1 Transport Layer Security

Il protocollo TLS (Transport Layer Security) è lo standard de facto per la protezione delle comunicazioni su reti non affidabili. La versione corrente, TLS 1.3, definita nella RFC 8446, introduce significativi miglioramenti rispetto alle versioni precedenti: eliminazione di suite crittografiche deboli, riduzione della latenza dell'handshake da due round-trip a uno, e forward secrecy obbligatoria attraverso l'uso esclusivo di scambi di chiave effimeri basati su Diffie-Hellman.

Il processo di handshake TLS stabilisce un canale sicuro tra client e server attraverso una sequenza di messaggi che negozia le suite crittografiche, autentica le parti e deriva le chiavi di sessione. Il risultato è un canale che garantisce confidenzialità (attraverso la cifratura simmetrica), integrità (attraverso MAC o AEAD) e autenticazione (attraverso certificati digitali).

Nel progetto, Envoy termina le connessioni TLS in ingresso, gestendo il processo di handshake e l'estrazione dei metadati crittografici attraverso il TLS Inspector. L'uso di TLS 1.3 è rilevante anche per il JA3 fingerprinting: client che negoziano con cipher suite o versioni TLS datate producono hash JA3 distinti e riconoscibili, consentendo al sistema di identificare connessioni anomale anche in presenza di credenziali valide.

2.4.2 Mutual TLS

Il mutual TLS (mTLS) è un'estensione del protocollo TLS in cui entrambe le parti, client e server, si autenticano reciprocamente mediante certificati digitali X.509. Nel TLS standard, solo il server presenta un certificato per autenticarsi al client; nell'mTLS, anche il client è tenuto a presentare un certificato valido, firmato da una Certificate Authority (CA) fidata dal server.

Questo meccanismo è particolarmente adatto agli ambienti Zero Trust, perché fornisce un'autenticazione forte basata su crittografia a chiave pubblica, indipendente da credenziali riutilizzabili come username e password. Un client che non disponga di un certificato valido firmato dalla CA interna non può stabilire una connessione, indipendentemente da qualsiasi altra credenziale.

Nel progetto, Envoy è configurato con `require_client_certificate: false`, rendendo facoltativa la presentazione di un certificato client firmato dalla CA interna. I certificati sono generati tramite OpenSSL con il parametro di estensione `extendedKeyUsage=clientAuth`, garantendo che possano essere utilizzati esclusivamente per l'autenticazione del client. Al termine dell'handshake, quando un certificato è presente, Envoy ne inietta i metadati nell'header `X-Forwarded-Client-Cert`; il Security Orchestrator li interpreta per risolvere l'identità del soggetto richiedente in modo crittograficamente verificato e li inoltra al livello di policy.

2.4.3 JA3 fingerprinting

Il JA3 fingerprinting è una tecnica di identificazione del client basata sui parametri dell'handshake TLS. Sviluppata da John Althouse, Jeff Atkinson e Josh Atkins presso Salesforce nel 2017, costruisce una firma identificativa del client a partire da cinque campi dell'handshake: la versione TLS proposta, le cipher suite supportate, le estensioni TLS incluse, i gruppi di curve ellittiche e i formati di punto ellittico.

Questi cinque campi vengono concatenati in una stringa con separatori definiti e poi sottoposti a hashing MD5, producendo un identificatore di 32 caratteri esadecimali che è caratteristico della combinazione di libreria TLS e configurazione utilizzata dal client. Client

diversi, anche se usano le stesse credenziali, produrranno hash JA3 differenti se il loro stack TLS è diverso. Questo rende il JA3 uno strumento utile per rilevare impersonificazioni, bot e client anomali anche quando le credenziali presentate sono valide.

Nel progetto, Envoy è configurato con `enable_ja3_fingerprinting: true` nel filtro TLS Inspector. Envoy espone la stringa JA3 grezza come variabile di metadato e la inietta negli header della richiesta inoltrata al Security Orchestrator. È quest'ultimo a calcolarne il digest MD5, ottenendo l'hash a 32 caratteri esadecimali.

2.5 Componenti tecnologici

2.5.1 nftables

nftables è il framework di filtraggio dei pacchetti del kernel Linux, introdotto come successore di iptables a partire dal kernel 3.13. A differenza di iptables, nftables offre una sintassi unificata per IPv4 e IPv6, supporto nativo per set e mappe di indirizzi, e un'architettura interna più efficiente basata su una macchina virtuale integrata nel kernel.

Nel progetto, nftables opera come firewall di rete posizionato tra il client esterno e il resto dell'infrastruttura. La sua configurazione definisce una tabella `inet filter` le cui tre catene principali, `input`, `forward` e `output`, adottano tutte una policy di default DROP, così che solo il traffico esplicitamente consentito venga accettato in ciascuna direzione. In particolare la catena `output` ammette il solo traffico in uscita strettamente necessario, ossia la risoluzione DNS e la comunicazione verso i microservizi interni. La catena `input` accetta il traffico locale, il traffico appartenente a connessioni già stabilite e il traffico TCP verso la porta di Envoy da indirizzi non presenti nella lista di blocco, loggando ogni pacchetto accettato o rifiutato tramite il gruppo NFLOG 100.

2.5.2 Snort

Snort è un sistema di rilevamento e prevenzione delle intrusioni di rete (NIDS/NIPS) open source, originariamente sviluppato da Marty Roesch nel 1998 e successivamente acquisito da Cisco. Opera analizzando il traffico di rete in tempo reale, confrontando i pacchetti catturati con un insieme di regole configurabili per rilevare pattern corrispondenti ad attività malevole note.

Snort può operare in tre modalità: sniffer (stampa i pacchetti sulla console), packet logger (salva i pacchetti su disco) e network intrusion detection system (analizza i pacchetti e genera alert). Nel progetto, Snort è configurato in modalità NIDS con output su console, condivide lo stack di rete del container firewall tramite la direttiva `network_mode: service:firewall`, e monitora l'interfaccia eth0.

La regola custom configurata utilizza il preprocessore `sfportscan` per rilevare scansioni di porte, abilitato con il parametro `sense_level: low` per minimizzare i falsi positivi. Una regola Snort aggiuntiva segnala esplicitamente la rilevazione di un port scan TCP quando vengono rilevati più di cinque pacchetti SYN dalla stessa sorgente nell'arco di

cinque secondi. Questo scenario è uno dei vettori di attacco testati esplicitamente nel progetto, con client di test che simulano la scansione rapida di un intervallo di porte.

2.5.3 Envoy Proxy

Envoy è un proxy L4/L7 open source sviluppato originariamente da Lyft e successivamente donato alla Cloud Native Computing Foundation (CNCF). È progettato per essere il piano dati di architetture a microservizi, gestendo la comunicazione tra servizi con funzionalità avanzate di load balancing, circuit breaking, osservabilità e sicurezza.

Nel contesto del progetto, Envoy svolge il ruolo di Policy Enforcement Point. La sua configurazione YAML definisce un listener sulla porta 8443 che, grazie al filtro `tls_inspector`, ispeziona i parametri dell'handshake TLS prima ancora che la connessione sia completamente stabilita. Il filtro `http_connection_manager` gestisce la connessione HTTP una volta che l'handshake TLS è concluso. Envoy è configurato per richiedere il certificato client e per loggare in formato JSON strutturato una serie di informazioni sulla connessione, tra cui la versione TLS, la cipher suite, l'indirizzo remoto e l'hash JA3, propagando l'header `X-Request-ID` per garantire la tracciabilità.

Il protocollo `ext_authz` di Envoy permette di delegare la decisione di autorizzazione a un servizio esterno, in questo caso il Security Orchestrator, prima di instradare la richiesta verso il backend. Questo è il meccanismo attraverso cui si realizza la separazione tra PEP e PDP.

2.5.4 Open Policy Agent e Rego

Open Policy Agent (OPA) è un motore di policy general-purpose open source, mantenuto dalla CNCF, che permette di esprimere e valutare policy in modo unificato attraverso diversi domini applicativi. OPA riceve in input un documento JSON che rappresenta il contesto della richiesta e valuta le policy scritte in Rego, restituendo un documento JSON che rappresenta la decisione.

Rego è un linguaggio dichiarativo derivato da Datalog, progettato specificamente per la definizione di policy. A differenza di un linguaggio imperativo, in Rego si descrive *cosa* deve essere vero perché una policy sia soddisfatta, non *come* verificarlo. Questo rende le policy leggibili, componibili e verificabili in modo indipendente rispetto al codice applicativo che le esegue.

Nel progetto, OPA opera come Policy Decision Point puro. Riceve dall'Orchestratore un payload che contiene : ruolo dell'utente, risorsa richiesta, metodo e score del rischio generato dal modello di AI Opa valuta tutte queste condizioni, come descritto in precedenza nella sezione 2.2, restituendo un verdetto binario allow/deny. La separazione tra la policy e il codice che la applica garantisce che le regole possano essere modificate, testate e verificate in modo indipendente dalla logica applicativa.

2.5.5 Splunk

Splunk è una piattaforma di analisi operativa dei dati macchina, ampiamente utilizzata come SIEM (Security Information and Event Management) in ambito enterprise. Raccoglie log da sorgenti eterogenee, li normalizza, li indicizza e li rende ricercabili e correlabili attraverso il linguaggio di query SPL (Search Processing Language).

Nel progetto, Splunk opera come centro di osservabilità dell'intera architettura. Ogni microservizio invia i propri log a Splunk tramite l'HTTP Event Collector (HEC), un endpoint HTTP che accetta eventi in formato JSON autenticati tramite token. L'uso del formato JSON strutturato permette a Splunk di indicizzare automaticamente i campi dei log, rendendoli immediatamente disponibili per ricerche e correlazioni.

Il campo `X-Request-ID`, propagato da Envoy e ripreso da ogni microservizio lungo la catena di elaborazione, è il meccanismo fondamentale per la correlazione degli eventi. Dato un identificatore di richiesta, è possibile ricostruire in Splunk il percorso completo della transazione attraverso tutti i componenti: il log del firewall nftables, il log di accesso di Envoy con l'hash JA3 e il codice di risposta, il log del Security Orchestrator con l'esito della valutazione OPA e le eventuali ragioni di deny, e il log del servizio di Business Logic.

2.5.6 MongoDB

MongoDB è un database orientato ai documenti, appartenente alla famiglia NoSQL, che archivia i dati in formato BSON (Binary JSON). La sua flessibilità dello schema e il supporto nativo per documenti annidati lo rendono particolarmente adatto a modellare strutture dati eterogenee e ricche di contesto come quelle richieste da un'architettura Zero Trust.

Nel progetto sono presenti due istanze MongoDB con ruoli distinti. Il Security DB conserva i dati di identità e dispositivo gestiti dall'iam-service e dal Security Orchestrator: le credenziali degli utenti con ruolo, livello di clearance e flag di enrollment TPM (collection `identity_users`), le credenziali WebAuthnFIDO2 dei dispositivi registrati (`device_fingerprints`), lo stato effimero delle sessioni OTP (`otp_sessions`), la blocklist dei token JWT revocati (`jwt_blocklist`), lo storico immutabile degli eventi di autenticazione (`auth_events`) e i contatori per il rate limiting (`rate_limits`). Il Business DB archivia i dati applicativi della centrale nucleare, organizzati in sette collection: `personnel`, `access-badges`, `zones`, `reactor-parameters`, `maintenance-orders`, `documents` e `nuclear-materials`.

2.5.7 Docker e Docker Compose

Docker è una piattaforma di containerizzazione che permette di eseguire applicazioni in ambienti isolati e riproducibili. Ogni container condivide il kernel del sistema operativo host ma dispone di un proprio filesystem, spazio di rete e namespace di processi, garantendo un isolamento sufficiente per la segmentazione dei microservizi.

Docker Compose è lo strumento di orchestrazione usato per definire e gestire l'insieme dei container che compongono l'applicazione. Il file `docker-compose.yaml` del progetto definisce diciassette servizi: oltre ai componenti principali dell'architettura (`firewall`, `envoy`, `security-orchestrator`, `opa` e `business-logic`), comprende le tre istanze di rilevamento delle intrusioni (`snort`, `snort-internal` e `snort-mid`), il servizio di identità `iam-service` con il relativo server di posta di sviluppo `mailhog`, il microservizio di scoring `ai-inference`, i due database `business-db` e `security-db`, l'utility `seeder`, lo stack di osservabilità (`splunk` con l'universal forwarder `splunk-uf`) e il generatore di alert di test `alerts-tester`. La topologia di rete è definita attraverso quattro reti Docker isolate: Front-Net, che collega i client al firewall e al Security Orchestrator; Auth-Net, che connette il Security Orchestrator a OPA, al servizio di identità e alla Security DB; Back-Net, che connette esclusivamente Business Logic e Business DB; e Snort-Net, dedicata al trasporto degli alert dalle istanze Snort verso il Security Orchestrator. Questa segmentazione garantisce che nessun componente possa comunicare direttamente con un servizio al di fuori del proprio segmento di rete autorizzato, applicando la complete mediation a livello infrastrutturale.

2.5.8 Go

Go, o Golang, è un linguaggio di programmazione compilato e tipizzato staticamente, sviluppato da Google e rilasciato nel 2009. Le sue caratteristiche principali includono la semplicità del modello di concorrenza basato su goroutine e canali, la produzione di binari staticamente linkati senza dipendenze runtime, e una libreria standard ricca che include primitive per la gestione di connessioni TCP, HTTP e TLS.

I servizi applicativi del progetto, Security Orchestrator e Business Logic, sono scritti in Go. Le linee guida di sviluppo interne al progetto prescrivono la propagazione del contesto tramite `context.Context` in tutte le chiamate che coinvolgono I/O, il wrapping degli errori con `fmt.Errorf` per preservare la traccia completa, e l'inclusione del `X-Request-ID` in tutti i log per garantire la tracciabilità. I client di test, scritti sia in Go sia in Python, simulano scenari di accesso legittimo, accesso anomalo con cipher suite deprecate e scansione delle porte, fornendo la base per la validazione dell'architettura.

2.5.9 Protobuf e gRPC

Protocol Buffers (protobuf) è un formato di serializzazione binaria sviluppato da Google, usato per definire strutture dati e interfacce di servizio in modo indipendente dal linguaggio. gRPC è il framework di Remote Procedure Call costruito su protobuf, che utilizza HTTP/2 come protocollo di trasporto e garantisce una comunicazione tipizzata, efficiente e indipendente dal linguaggio tra servizi.

Nel progetto, il file `ext_authz.proto` definisce formalmente l'interfaccia del Security Orchestrator, con un metodo `Check` che accetta un `CheckRequest` contenente l'identificatore della richiesta e restituisce un `CheckResponse` con il campo booleano `allowed`. Questo file rappresenta il contratto architetturale tra Envoy e il Security Orchestrator. L'implemen-

tazione corrente del Security Orchestrator espone l'endpoint `/api/v1/evaluate` tramite HTTP, in modo funzionalmente equivalente a quanto definito nel contratto proto; la migrazione a gRPC nativo costituisce una possibile evoluzione futura che consentirebbe di sfruttare appieno le garanzie di tipizzazione e le prestazioni di HTTP/2.

3 Architettura del sistema

L'architettura di ZTALeaks si fonda su un principio fondamentale: nessun elemento interno riceve fiducia implicita e ogni componente opera con responsabilità rigidamente confinate. Il sistema implementa concretamente la separazione tra Policy Enforcement Point (PEP) e Policy Decision Point (PDP) prescritta dalle linee guida dello standard NIST SP 800-207. Questa separazione viene realizzata tramite un ecosistema di microservizi isolati a livello di rete e containerizzati, i quali cooperano attraverso scambi protetti e log tracciabili in modo centralizzato.

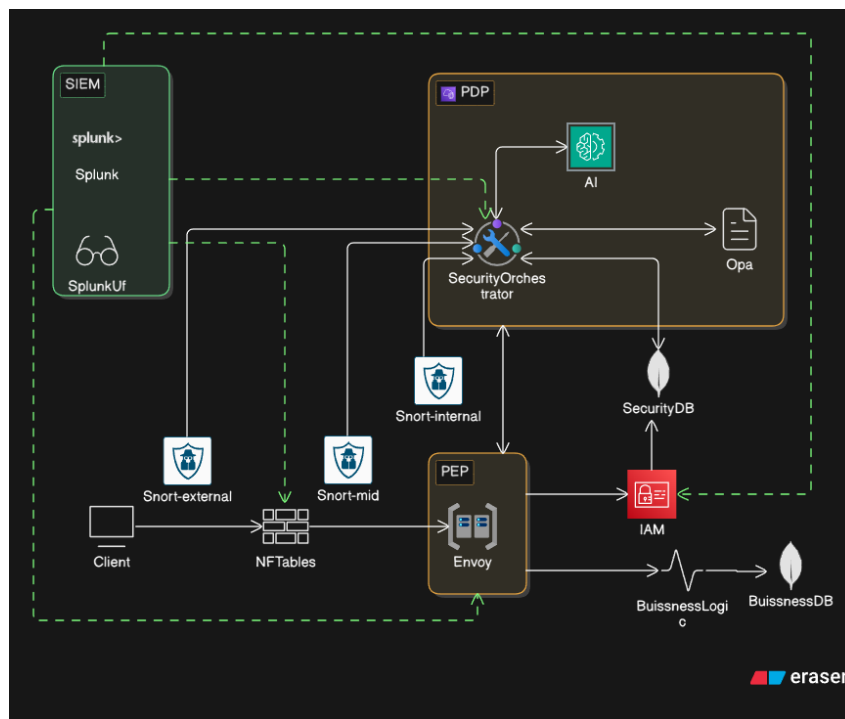


Figura 3.1: Immagine architettura

3.1 Struttura della repository

Il codice sorgente dell'infrastruttura è strutturato per rispecchiare la scomposizione logica del sistema. Le cartelle principali che compongono il progetto sono descritte nell'elenco seguente:

- `api/`: ospita il contratto protocollare `ext_authz.proto`, che costituisce la definizione formale dell'interfaccia di comunicazione tra i sistemi;

- **certs/**: raccoglie i certificati digitali e le chiavi private impiegati per la mutua autenticazione del canale cifrato, includendo i certificati della Certificate Authority interna e lo script di generazione `gen-certs.sh`;
- **deployments/**: racchiude le configurazioni per il deployment dell'infrastruttura, suddividendosi in **docker/**, che contiene il file `docker-compose.yaml` per l'avvio ordinario e `docker-compose.test.yaml` per i test, e **kubernetes/**, che contiene le definizioni per l'orchestrazione dei pod;
- **docs/**: contiene la documentazione tecnica e le guide di riferimento per l'attivazione e la manutenzione dell'infrastruttura;
- **infra/**: ospita i file di configurazione dei servizi infrastrutturali, strutturandosi in **databases/** per MongoDB, **envoy/** per il proxy, **nftables/** per il firewall, **opa/** per le regole di accesso e **snort/** con **snort-internal/** per il rilevamento delle intrusioni;
- **services/**: comprende le implementazioni software dei servizi attivi, includendo **business-logic/** scritto in Go per la gestione dei dati applicativi, **frontend/** con le pagine statiche e l'interfaccia utente, e **security-orchestrator/** scritto in Go come coordinatore decisionale;
- **tasks/**: contiene script e automazioni per l'esecuzione di operazioni periodiche e di manutenzione sul sistema;
- **tests/**: raccoglie gli strumenti per la convalida dei componenti, comprendendo la cartella **clients/** con simulatori in Python e Go per la generazione di traffico e la cartella **dashboard-app/** per la visualizzazione dei log;
- **tools/**: ospita utilità di supporto tra cui il seeder del database per la generazione di record coerenti.

La radice della repository ospita il file `.env` con le chiavi di configurazione necessarie, il file `.gitignore` e il file `README.md`.

3.2 La segmentazione di rete

La topologia di rete si articola in tre segmenti Docker isolati, applicando il controllo degli accessi a livello infrastrutturale. Questa compartimentazione assicura che un compromesso su un servizio non si propaghi ad altri domini.

La **Front-Net** rappresenta l'unico canale rivolto all'esterno. In questo segmento risiedono il firewall di rete, il proxy Envoy e il Security Orchestrator. Qualsiasi richiesta in ingresso deve attraversare questo canale, venendo sottoposta ai controlli del firewall prima di poter raggiungere i servizi interni.

La **Auth-Net** costituisce il segmento privato dedicato alla determinazione delle autorizzazioni. In questa rete cooperano il Security Orchestrator, il motore Open Policy Agent e il database di sicurezza. Il Security Orchestrator funge da unico elemento di connessione

tra la **Front-Net** e la **Auth-Net**, impedendo la raggiungibilità diretta del database di sicurezza e del motore delle regole da parte dei client esterni.

La **Back-Net** è una rete protetta e riservata che unisce esclusivamente il servizio di business logic al relativo database di produzione. Questa separazione assicura che il database contenente i dati sensibili della centrale sia isolato sia dall'esterno sia dallo stesso piano di decisione delle policy, riducendo l'esposizione a tentativi di esfiltrazione. Il servizio Business Logic appartiene sia alla **Back-Net** sia alla **Auth-Net**, il che consente di esporre l'endpoint interno `/internal/session` al Security Orchestrator per l'arricchimento del contesto di autorizzazione, senza però esporre il database sottostante.

3.3 Il firewall e il punto di ingresso

Il container denominato `firewall` costituisce la barriera iniziale per il traffico in ingresso. Configurato tramite `nftables` su una distribuzione Alpine Linux, il firewall adotta una politica predefinita di tipo **DROP** per le catene di ingresso, transito e uscita.

Per impedire che il traffico possa aggirare i controlli, il proxy Envoy e i sistemi di rilevamento delle intrusioni Snort condividono lo stack di rete del firewall tramite la configurazione `network_mode` di Docker. Questa unione assicura che ogni pacchetto debba superare i controlli a livello kernel prima di essere intercettato da Envoy o ispezionato da Snort.

La configurazione del firewall include regole per contrastare gli attacchi volumetrici di tipo SYN flood tramite una struttura dinamica denominata `syn_meter`, la quale limita a venti pacchetti al secondo la frequenza per ogni indirizzo IP sorgente, respingendo le eccedenze con il log di avviso `fw-syn-flood-drop`. Inoltre, la catena di uscita implementa un filtraggio rigoroso che consente le sole comunicazioni dirette alle porte TCP di risoluzione dei nomi e ai microservizi interni, ossia le porte 53, 8080, 8081 e 8082, scartando ogni altra richiesta in uscita sotto il log `fw-egress-drop`. Il demone `ulogd` raccoglie questi eventi tramite `NFLOG` e li scrive su un file in emulazione `syslog`. Un parser scritto in Go legge questo file e genera un log JSONL strutturato nel percorso `/var/log/ztaleaks/nftables/firewall.jsonl`, rendendolo disponibile per l'inoltro e l'indicizzazione centralizzata.

3.4 Envoy Proxy come Policy Enforcement Point

Il proxy Envoy agisce come Policy Enforcement Point del sistema, operando sulla porta 8443. Il listener impiega il filtro `tls_inspector` per analizzare l'handshake in modo precoce, estraendo l'impronta JA3 del client e rendendola disponibile nei metadati di sessione.

Il canale TLS è configurato per richiedere la presentazione facoltativa di un certificato client, demandando la verifica del possesso e la validazione del certificato al livello applicativo. Envoy utilizza il filtro HTTP `ext_authz` configurato tramite il blocco `http_service`

per inoltrare in modo sincrono i dettagli della richiesta al Security Orchestrator sull'endpoint `/api/v1/evaluate` tramite una chiamata HTTP. La direttiva `failure_mode_allow` è impostata su `false`, applicando una modalità di blocco a fronte di un eventuale guasto del coordinatore.

Le informazioni estratte dal certificato client e l'impronta JA3 vengono inserite negli header `x-forwarded-client-cert` e `x-ja3-fingerprint` prima di inviare la richiesta all'orchestratore. I log di accesso di Envoy sono scritti in formato JSONL includendo il valore letterale `service: "envoy"`, mentre i campi relativi a punteggi di rischio esterni o identificativi di zona geografica sono momentaneamente omessi in quanto non popolati dalle logiche attuali del sistema.

3.5 Il Security Orchestrator come Policy Administrator

Il Security Orchestrator opera come Policy Administrator, ricevendo le richieste di valutazione dal proxy Envoy. Questo microservizio, sviluppato in Go, espone un server HTTP sulla porta 8081 e coordina le componenti coinvolte nel processo decisionale.

All'avvio, il servizio configura un logger JSON strutturato che integra il valore `service: "security-orchestrator"`. Per ogni richiesta ricevuta sulla rotta `/api/v1/evaluate`, l'orchestratore esegue le seguenti attività:

- estrae il token JWT dall'header di autorizzazione e ne convalida la firma crittografica tramite l'endpoint JWKS esposto dall'identity service;
- identifica il dispositivo analizzando l'header del certificato mTLS e interroga il database di sicurezza per verificare la validità dell'attestazione TPM associata al dispositivo;
- esegue un controllo in modalità trasparente, denominata Shadow Mode, confrontando l'unità organizzativa del certificato con il ruolo dell'utente ed evidenziando eventuali disallineamenti;
- arricchisce il contesto con il punteggio di rischio calcolato da un modello di Intelligenza Artificiale di tipo Temporal Graph Network (TGN), interrogato come microservizio esterno: il modello ricostruisce il grafo storico delle interazioni e restituisce un *anomaly score* continuo nell'intervallo $[0, 1]$. Per resistere ad attacchi di tipo *poisoning*, l'interrogazione segue un flusso in due fasi: l'orchestratore chiama prima l'endpoint `/infer` in sola lettura per ottenere lo score e, solo se la decisione finale di OPA è di consenso, conferma l'evento nella memoria del modello tramite l'endpoint `/update`, così che gli accessi negati non contribuiscano mai all'addestramento. In caso di indisponibilità o timeout del servizio (cinque secondi), l'orchestratore adotta in modo conservativo uno score elevato pari a 0.99 con confidenza `low`, traducendo l'incertezza in una postura più restrittiva;
- compone un documento strutturato e interroga il motore OPA per la decisione definitiva.

Le decisioni emesse vengono registrate in un log dedicato denominato `opa_decision.jsonl` per finalità di ispezione e analisi forense.

3.5.1 Alert provenienti dagli Snort

Parallelamente al flusso di valutazione sincrono, il Security Orchestrator mantiene una cache in memoria degli alert generati dalle istanze Snort. Un listener TCP dedicato, in ascolto sulla porta 9000, riceve dal parser degli IDS gli allarmi serializzati come JSON newline-delimited e li indicizza per indirizzo IP sorgente. Per ciascun IP la cache conserva tre slot distinti, corrispondenti alle istanze *edge* (`snort`), *mid* (`snort-mid`) e *internal* (`snort-internal`), associati a una scadenza temporale (TTL) di tre minuti che viene rinnovata a ogni nuovo allarme. Al momento della valutazione di una richiesta, l'orchestratore interroga la cache per l'IP del client e converte la presenza di allarmi recenti in feature binarie, passate al modello TGN insieme agli altri attributi della richiesta (corrispondenza del fingerprint JA3, metodo, ruolo e livello di clearance). In questo modo un'attività di rete sospetta rilevata da Snort innalza in tempo reale lo score di rischio associato alle richieste provenienti dal medesimo indirizzo, senza introdurre latenza aggiuntiva nel percorso decisionale.

3.6 OPA come Policy Decision Point

La policy predefinita applica un approccio *default deny*, bloccando qualsiasi richiesta non esplicitamente autorizzata. La valutazione autorizzativa si articola ad alto livello su una catena di controlli logici sequenziali: la verifica di eventuali esenzioni per le rotte pubbliche, l'assegnazione degli attributi di sicurezza in base all'identità (RBAC), la validazione matematica dell'appartenenza ai compartimenti richiesti (Categorie) e il controllo gerarchico di confidenzialità basato sulle direttive del modello Bell-LaPadula.

A valle dei controlli strutturali, la policy integra la valutazione continua del rischio generata in tempo reale dai sistemi di Intelligenza Artificiale. Invece di affidarsi a soglie di blocco fisse o a meccanismi di fallback statici, il sistema valuta ogni singola transazione attraverso il calcolo di un bilancio di rischio netto: l'accesso viene definitivamente autorizzato solo se il beneficio intrinseco dell'operazione, decurtato dello score di minaccia calcolato dall'AI, risulta superiore al livello di rischio accettato per quello specifico endpoint.

3.7 Il Business Logic service

Il servizio Business Logic rappresenta il backend applicativo che gestisce il dominio operativo della centrale. Il servizio è scritto in Go e popola i propri log integrando il campo `service`: `"business-logic"` per abilitare la correlazione automatica in Splunk.

Un middleware intercetta le richieste in transito, estraendo gli header di tracciamento iniettati da Envoy come `X-Request-ID`, `X-Current-User` e `X-Ja3-Fingerprint`. Questi

dati vengono combinati con la durata e lo stato della risposta per produrre un record di log esaustivo.

Per garantire la protezione contro tentativi di manomissione diretta della base dati, il servizio implementa una convalida dell'integrità: all'atto della creazione o dell'aggiornamento di un record sensibile, il sistema calcola un hash SHA-256 concatenando i campi informativi con un carattere di separazione pipe; questo hash viene memorizzato nel documento e ricalcolato a ogni lettura, bloccando la visualizzazione e sollevando un errore qualora si rilevi una divergenza.

3.8 Identity Access Management service

Il servizio Identity Access Management (`iam-service`) gestisce l'intero ciclo di vita dell'autenticazione e dell'identità. È scritto in Go, espone un server HTTP sulla porta 8082 e popola i propri log integrando il campo `service`: `"iam-service"` per la correlazione automatica in Splunk. L'autenticazione segue un flusso a due fasi che incorpora un secondo fattore obbligatorio. Nella prima fase, la rotta `/api/v1/auth/login` verifica le credenziali confrontando la password con un hash `Argon2id` e controlla il certificato client propagato da Envoy controllando il campo CN deve corrispondere allo username e l'unità organizzativa al ruolo dichiarato. Superati questi controlli, il servizio genera un codice OTP a sei cifre, ne memorizza l'hash HMAC-SHA256 con una scadenza di cinque minuti e un massimo di tre tentativi, e lo invia all'indirizzo email registrato. Nella seconda fase, la rotta `/api/v1/auth/verify-otp` valida il codice e, solo in caso di esito positivo, rilascia il token di sessione. Il token emesso è un JWT firmato con algoritmo RS256, con chiave pubblica RSA corrispondente è pubblicata sull'endpoint `/.well-known/jwks.json`, da cui il Security Orchestrator la recupera per verificare in autonomia la firma di ogni token, senza dover condividere segreti con l'`iam-service`.

Il servizio gestisce inoltre l'elevazione del livello di fiducia del dispositivo. Attraverso le cerimonie WebAuthn/FIDO2 un utente può registrare un dispositivo tramite TPM: la credenziale viene salvata nella collezione `device_fingerprints`,

3.9 I database

L'infrastruttura utilizza due database MongoDB separati per impedire la commistione tra dati di sicurezza e informazioni di produzione.

Il Business DB memorizza i dati del personale e della centrale. Il database abilita l'autenticazione obbligatoria e definisce quattro utenze a minimo privilegio, assegnando permessi distinti per le operazioni di scrittura applicativa, per il caricamento iniziale dei dati, per il motore PDP e per i lettori di monitoraggio. Ciascuna delle sette collezioni è protetta da un JSON Schema che valida i tipi di dato e la coerenza dei formati. I documenti del personale e delle zone includono rispettivamente i sotto-documenti `ztna_metadata` e `ztna_policy` per supportare le verifiche del Policy Decision Point.

Il Security DB è destinato esclusivamente al salvataggio delle impronte digitali dei dispositivi registrati nella collezione `device_fingerprints` e degli utenti abilitati nel servizio di identità, rimanendo accessibile solo all'orchestratore e al servizio di identità.

3.10 Il sistema di rilevamento delle intrusioni

Il monitoraggio della sicurezza di rete si avvale di tre istanze distinte di Snort, posizionate strategicamente all'interno dell'infrastruttura per ispezionare il traffico a diversi livelli:

- **Istanza Esterna (`snort`):** Analizza il traffico in ingresso sulla `Front-Net` e si concentra sull'individuazione di scansioni di porte tramite il modulo `sfportscan`, rilevando attività sospette a fronte di cinque pacchetti SYN ravvicinati provenienti dal medesimo mittente.
- **Istanza Interna (`snort-internal`):** Analizza le transazioni interne dirette verso il proxy Envoy, cercando violazioni del canale TLS. Questo comprende l'individuazione di negoziazioni prive di certificato client, l'impiego di cifrari obsoleti o connessioni basate su versioni non sicure del protocollo.
- **Istanza Mid (`snort-mid`):** Ispeziona il traffico applicativo in transito tra Envoy e il Security Orchestrator (sul canale `ext_authz`). Questa istanza è specificamente configurata per rilevare e segnalare vettori di attacco di livello 7, come tentativi di SQL Injection e Cross-Site Scripting (XSS).

3.10.1 Meccanismo di Inoltro degli Alert

Affinché le minacce vengano gestite tempestivamente, le istanze IDS sono collegate a un sistema di telemetria in tempo reale. Un parser personalizzato scritto in Go legge i log generati dalle tre istanze di Snort in modalità *tailing*, normalizzandoli in formato JSONL.

Una volta normalizzato, ogni evento critico viene immediatamente inoltrato al Security Orchestrator attraverso una connessione socket TCP diretta (sulla porta 9000). Questo scambio avviene su una porzione di rete privata dedicata esclusivamente al trasporto degli allarmi (`snort-net`). L'utilizzo del socket TCP permette un invio a bassissima latenza, garantendo che gli eventi di sicurezza raggiungano il motore decisionale centrale nel momento esatto in cui vengono rilevati sul livello di rete, pronti per l'indicizzazione e per innescare eventuali risposte di mitigazione.

3.11 Il frontend

L'interfaccia utente è costituita da un'applicazione web statica servita da Nginx. L'interfaccia comunica direttamente con l'API esposta da Envoy sulla porta 8443.

Il frontend gestisce in modo autonomo il token di autenticazione e richiede un token CSRF prima di intraprendere operazioni di scrittura. Sebbene la sicurezza sia garantita dal controllo delle policy eseguito dal PDP, il frontend implementa regole lato client per

nascondere o disattivare elementi grafici non conformi ai privilegi dell'utente, migliorando la qualità dell'interazione.

3.12 Il seeder

Il seeder è uno strumento sviluppato in Go per popolare il Business DB. Per garantire la coerenza, l'applicazione inserisce i record rispettando rigorosamente le dipendenze referenziali tra le collezioni.

Il seeder calcola preventivamente gli hash di integrità crittografica per i record inseriti, rendendo l'ambiente immediatamente verificabile e pronto all'uso fin dalla prima inizializzazione.

3.13 Il flusso di autorizzazione end-to-end

La cooperazione tra i componenti può essere compresa seguendo il percorso di una transazione, illustrato nella catena di passaggi descritta di seguito:

- il client effettua la chiamata verso la porta **8443** di Envoy, attraversando le regole del firewall che verificano l'assenza dell'indirizzo sorgente dalla lista nera e valutano i limiti di frequenza;
- Envoy instaura la sessione TLS raccogliendo i dati del certificato client e l'impronta JA3, inviando poi una chiamata HTTP al Security Orchestrator con i dettagli della transazione;
- il Security Orchestrator valida il token JWT tramite JWKS, verifica l'attestazione TPM sul database di sicurezza e arricchisce la richiesta con il punteggio del rischio calcolato dai sistemi di Intelligenza Artificiale;
- OPA riceve l'input arricchito dall'orchestratore, valuta la policy multi-dimensionale e restituisce il verdetto finale;
- se OPA consente l'accesso, l'orchestratore autorizza Envoy, il quale inoltra la richiesta al Business Logic; quest'ultimo convalida l'integrità dei record tramite SHA-256 e risponde al client includendo l'identificativo **X-Request-ID** per la correlazione centralizzata dei log.

4 Implementazione dei componenti

Dopo aver delineato nel capitolo precedente la struttura generale dell'architettura e il flusso di autorizzazione end-to-end, l'attenzione viene ora focalizzata sull'analisi dettagliata di ciascun componente. L'obiettivo di questo capitolo è esaminare in profondità il funzionamento interno di ogni modulo del sistema, illustrando le scelte implementative, i meccanismi crittografici, le strutture dati e le interazioni reciproche tra i servizi. Si procede seguendo il percorso che una richiesta compie attraversando l'infrastruttura, partendo dal firewall di rete e giungendo fino al database applicativo, in modo da costruire una comprensione progressiva e coerente dell'intero sistema.

4.1 Il firewall di rete con nftables

Il primo componente che intercetta il traffico in ingresso è il container `ztaleaks_firewall`, costruito su un'immagine Alpine Linux minimale e configurato per eseguire `nftables` come sistema di filtraggio dei pacchetti. Questo container svolge un duplice ruolo nell'architettura: da un lato applica una policy di filtraggio rigorosa sul traffico in ingresso, dall'altro fornisce lo stack di rete condiviso per i container Envoy e le tre istanze Snort, che sono configurati con la direttiva `network_mode:service:firewall` nel file di Compose.

Questa scelta architetturale ha conseguenze rilevanti sulla sicurezza. Il fatto che Envoy e le istanze Snort condividano il namespace di rete del firewall significa che ogni pacchetto destinato a Envoy attraversa obbligatoriamente le catene di nftables prima di raggiungere il proxy, e che Snort osserva esattamente lo stesso flusso di traffico che Envoy riceve. Non esiste alcun percorso alternativo: la complete mediation è garantita a livello del kernel Linux, senza affidarsi a configurazioni applicative che potrebbero essere aggirate.

La configurazione di `nftables` è definita nel file `nftables.conf` montato all'interno del container. La tabella principale è denominata `inet filter` e gestisce sia il traffico IPv4 sia quello IPv6 in modo unificato. All'interno della tabella vengono definite cinque catene: tre catene IDS dedicate al dirottamento del traffico verso le istanze Snort tramite NFQUEUE, e le catene tradizionali `input`, `forward` e `output`.

Catene IDS con NFQUEUE

Prima di raggiungere le catene di filtraggio, il traffico transita attraverso tre catene IDS configurate con priorità negative, in modo da eseguire l'ispezione Snort prima di qualsiasi decisione di filtraggio. La direttiva `bypass` garantisce il comportamento fail-open: se un'istanza Snort si arresta, il kernel accetta il pacchetto invece di scartarlo, mantenendo operativo il sistema. Le istanze Snort operano in modalità pura IDS, emettendo solo verdetti `ACCEPT`, senza mai bloccare il traffico autonomamente.

Le tre catene IDS sono le seguenti:

- La catena `ids_prerouting`, con priorità `-150`, intercetta tutto il traffico in ingresso sull'interfaccia di rete esterna (`front-net`) e lo dirige verso la coda numero 1, dove opera l'istanza Snort esterna che rileva attività di scansione e ricognizione.
- La catena `ids_prerouting_tls`, con priorità `-140`, intercetta il solo traffico TCP diretto alla porta di Envoy e lo dirige verso la coda numero 2, dove opera l'istanza Snort interna che analizza le anomalie TLS e i tentativi di SYN flood.
- La catena `ids_output_extauthz`, con priorità `-150` sul hook `output`, intercetta il traffico TCP diretto alla porta 8081 del Security Orchestrator e lo dirige verso la coda numero 3, dove opera l'istanza Snort intermedia che rileva attacchi applicativi (SQL injection e XSS) sul canale `ext_authz` in chiaro.

Catene di filtraggio

La catena `input` è configurata con policy di default DROP. Questa scelta rappresenta l'applicazione del principio di fail-safe defaults: in assenza di una regola esplicita di consenso, il pacchetto viene scartato silenziosamente. Le regole di consenso sono limitate al minimo necessario: viene accettato il traffico locale sull'interfaccia di loopback, il traffico appartenente a connessioni già stabilite o correlate (sfruttando il connection tracking del kernel) e il traffico TCP nuovo verso la porta di Envoy, loggato con il prefisso `fw-accept`.

Un meccanismo di rilievo è rappresentato dal controllo anti-SYN flood: tramite un contatore dinamico denominato `syn_meter`, le connessioni con flag SYN che superano la frequenza di venti richieste al secondo per singolo indirizzo IP sorgente vengono scartate e registrate nei log con il prefisso `fw-syn-flood-drop`.

La catena `output` implementa anch'essa una policy di default DROP per impedire connessioni non autorizzate verso l'esterno. La allow-list di uscita comprende il traffico di loopback, le connessioni già stabilite, le porte TCP dei servizi applicativi (8080, 8081, 8082), la porta del socket TCP Snort verso l'orchestratore (9000), le porte per la risoluzione DNS (53 TCP e UDP), la porta MongoDB (27017) e la porta syslog (1025). Ogni altra connessione in uscita viene rifiutata e registrata con il prefisso `fw-egress-drop`.

La catena `forward` adotta una policy di default DROP, prevenendo qualsiasi instradamento non esplicitamente previsto tra le interfacce del container.

Avvio e configurazione a runtime

Lo script `entrypoint.sh` orchestra l'avvio del container in sei fasi sequenziali: crea le directory di log, avvia `ulogd` in background, sostituisce nella configurazione le due variabili segnaposto `ENV_ENVOY_PORT` e `ENV_FRONT_IFACE` con i valori effettivi prelevati dall'ambiente (la seconda viene risolta dinamicamente interrogando la subnet `FRONT_NET_SUBNET` tramite il comando `ip`), carica le regole nel kernel tramite `nft delete table` seguito da `nft -f`, avvia il parser Go in background e infine mantiene il container attivo in foreground effettuando un tail del file di log.

Il logging è gestito dal demone `ulogd`, configurato con uno stack di plugin che riceve gli eventi dal gruppo NFLOG numero 100, li arricchisce con metadati di rete e li scrive sul file `ulogd-syslogemu.log`. Il parser Go (`parser.go`) legge tale file in modalità `tail -F` (resistente alla rotazione dei log), estrae i campi chiave-valore dal formato syslog, identifica il prefisso del log (es. `fw-accept`, `fw-syn-flood-drop`, `fw-egress-drop`) e produce record JSON strutturati con campi `action` e `threat`, scritti in append sul file `firewall.jsonl` all'interno del volume condiviso con il Splunk Universal Forwarder.

4.2 Il sistema di rilevamento intrusioni con Snort

Sul medesimo stack di rete del firewall operano tre container Snort distinti, ciascuno collegato a una coda NFQUEUE specifica e dedicato a un livello di ispezione differente. I tre container sono denominati `ztaleaks_snort` (coda 1), `ztaleaks_snort_internal` (coda 2) e `ztaleaks_snort_mid` (coda 3), e svolgono funzioni complementari nell'architettura di rilevamento.

Tutte e tre le immagini sono costruite con un processo multi-stage: il primo stage compila in Go il parser degli alert, mentre il secondo stage installa Ubuntu 24.04 con Snort 2.9 dai repository ufficiali e incorpora il binario compilato. Snort viene avviato con i flag `-k none` per disabilitare la verifica dei checksum (necessaria sulle interfacce virtuali Docker), `-A fast` per il formato di output degli alert, e `-daq nfq -daq-var queue=N` per l'integrazione NFQUEUE.

Istanza esterna (coda 1): scansioni di porte

Il container `ztaleaks_snort` monitora tutto il traffico in ingresso sull'interfaccia `front-net`. Il preprocessore `sfportscan` è abilitato con i parametri `proto {all}`, `memcap {10000000}` e `sense_level {low}` per minimizzare i falsi positivi. L'unica regola nel file `portscan.rules` (SID 1000001) genera l'alert *ZTA: TCP Port Scanning Detected* quando vengono rilevati più di cinque pacchetti TCP con flag SYN dalla stessa sorgente nell'arco di cinque secondi, con classificazione `attempted-recon`.

Istanza interna (coda 2): anomalie TLS

Il container `ztaleaks_snort_internal` monitora il traffico TCP diretto alla porta di Envoy, specializzandosi nell'ispezione dei record TLS. Il file `internal.rules` implementa un insieme articolato di regole suddivise in tre sezioni:

- *Anomalia mTLS*: la regola SID 2000002 individua la violazione mTLS cercando il pattern binario `0B 00 00 03 00 00 00`, corrispondente a un messaggio Certificate di tipo `HandshakeType=11` con lista di certificati vuota, indicativo dell'assenza del certificato client. Una regola ICMP canary (SID 1000004) viene usata durante i test di integrazione per verificare che l'IDS sia attivo.

- *Versioni TLS legacy*: tre regole (SID 2000010–2000012) rilevano l’offerta di SSLv3, TLS 1.0 e TLS 1.1 nel campo `legacy_version` del ClientHello (offset 9–10 del payload), scelto deliberatamente rispetto alla versione del record TLS (bytes 1–2) per evitare falsi positivi sulle connessioni TLS 1.3, che impostano la versione del record a 0x0301 per compatibilità. Tre ulteriori regole (SID 2000020–2000022) rilevano i corrispondenti attacchi di downgrade nel ServerHello.
- *Cipher suite deboli*: tre regole (SID 2000030–2000032) rilevano cipher suite RC4, 3DES e NULL/EXPORT/DH anonime offerte nel ClientHello, utilizzando espressioni regolari PCRE ancorate alla posizione relativa dopo l’intestazione del messaggio.
- *SYN flood*: la regola SID 2000006 genera un alert quando un singolo destinatario riceve più di trenta pacchetti SYN nell’arco di due secondi verso la porta di Envoy, con classificazione `attempted-dos`.

Istanza intermedia (coda 3): attacchi applicativi

Il container `ztaleaks_snort_mid` monitora il traffico TCP tra Envoy e il Security Orchestrator sulla porta 8081, ispezionando il canale `ext_authz` in chiaro. Il file `mid.rules` implementa due categorie di rilevamento:

- *SQL Injection* (SID 3000001–3000004): quattro regole rilevano rispettivamente il pattern `UNION SELECT`, la tautologia `OR 1=1`, l’istruzione distruttiva `DROP TABLE` e la tautologia URL-encoded `%27 OR ... %3D`.
- *Cross-Site Scripting* (SID 3000010–3000014): cinque regole rilevano i tag `<script>` in forma grezza e URL-encoded, lo schema URI `javascript:` e i gestori di eventi `onerror=` e `onload=`.

Parser e integrazione con l’orchestratore

Ogni istanza Snort esegue in background un parser Go condiviso (`parser.go`) che legge il file degli alert in formato `fast` e applica un’espressione regolare per estrarre i campi strutturati di ciascun alert: timestamp, messaggio, classificazione, priorità, indirizzo e porta sorgente, indirizzo e porta di destinazione. Gli alert strutturati vengono scritti come JSON su un file JSONL specifico del container e trasmessi in tempo reale al Security Orchestrator tramite una connessione TCP persistente sulla porta 9000, dove vengono inseriti nella `SnortCache` con TTL di tre minuti per alimentare la valutazione del rischio.

4.3 Envoy Proxy e la pipeline TLS

Concluso il livello di filtraggio e ispezione IDS, il traffico raggiunge Envoy, il componente più articolato dell’architettura dal punto di vista della configurazione. Il file `envoy.yaml`, montato nel container all’avvio, definisce in modo dichiarativo l’intera pipeline di terminazione TLS, intercezione delle richieste, delega dell’autorizzazione e instradamento verso i backend.

La sezione `static_resources` contiene la configurazione dei listener e dei cluster, ovvero rispettivamente i punti di ingresso del traffico e i gruppi di endpoint upstream verso cui le richieste vengono instradate.

Il listener principale, denominato `listener_0`, si pone in ascolto sulla porta `8443` e applica come primo filtro di livello listener il `tls_inspector`, configurato con la direttiva `enable_ja3_fingerprinting: true`. Questo filtro analizza i parametri dell'handshake TLS prima ancora che la negoziazione sia completata, calcolando l'impronta JA3 del client e rendendola disponibile come variabile di metadato attraverso il placeholder `%TLS_JA3_FINGERPRINT%`.

La catena di filtri prosegue con il `transport_socket` di tipo `DownstreamTlsContext`, che gestisce la terminazione TLS. La configurazione carica il certificato server, la chiave privata e il certificato della CA interna dai percorsi `/etc/envoy/certs/`. Il parametro `require_client_certificate` è impostato su `false` per consentire l'accesso iniziale agli utenti non ancora in possesso di un certificato client, lasciando al livello superiore la valutazione dei requisiti di sicurezza.

Completato l'handshake TLS, il controllo passa al filtro di rete `http_connection_manager`.

La direttiva `use_remote_address: true` abilita il popolamento dell'header `x-forwarded-for` con l'indirizzo IP reale del client downstream, con una configurazione esplicita degli intervalli di indirizzi interni per la corretta classificazione del traffico. La direttiva `forward_client_cert_details_sanitized: true` garantisce che l'header `x-forwarded-client-cert` venga sempre sovrascritto con il valore derivato dalla connessione TLS effettiva, prevenendo attacchi di spoofing.

La configurazione degli access log prevede un doppio output: uno verso lo standard output (per i log Docker) e uno verso il file `access.jsonl` all'interno del volume condiviso. Il formato dei log è JSON strutturato e include i campi per la correlazione: l'identificatore `request_id` (propagato dall'header `X-Request-ID`), l'impronta JA3, l'indirizzo remoto, il cluster upstream, il certificato client e l'identità dell'utente autenticato.

La sezione `request_headers_to_add` inietta automaticamente tre header nelle richieste: `x-ja3-fingerprint` con il valore dell'impronta JA3, `x-original-uri` con il path originale della richiesta e `x-envoy-external-address` con l'indirizzo IP esterno del client, derivato dall'indirizzo downstream reale e non falsificabile dal client.

Le rotte dell'identity access management (`/api/v1/auth/`, `/.well-known/jwks.json`, `/login`, `/register`) vengono instradate direttamente al cluster `identity_service_cluster` senza passare per il filtro `ext_authz`, poiché gestiscono richieste non autenticate. Tutte le altre rotte (prefisso `/api/v1/` e radice `/`) vengono instradate al cluster `business_logic_cluster` ma devono prima superare il filtro `ext_authz`.

La pipeline dei filtri HTTP comprende due elementi in sequenza. Il primo è il filtro `ext_authz`, che effettua una chiamata sincrona di autorizzazione al Security Orchestrator sull'endpoint `/api/v1/evaluate` con un timeout di cinque secondi. La direttiva `failure_mode_allow: false` garantisce il comportamento fail-closed (Zero Trust). Il filtro trasmette al Security Orchestrator un insieme definito di header tra cui

`authorization`, `x-forwarded-client-cert`, `x-ja3-fingerprint`, `x-original-uri`, `x-zone-id` e `x-envoy-external-address`, e riceve in risposta gli header `x-current-user`, `x-risk-score` e `x-envoy-external-address` che vengono propagati all'upstream. Il secondo filtro è il `router`, che completa l'instradamento.

4.4 Il Security Orchestrator

Il Security Orchestrator costituisce il punto di raccordo decisionale del sistema, scritto in Go e strutturato per operare come server HTTP sulla porta 8081. Il punto di ingresso si trova nel file `cmd/orchestrator/main.go`.

All'avvio, il programma configura un logger JSON strutturato che scrive sia su stdout sia su file (`app.jsonl`) all'interno del volume dedicato, con il valore di provenienza prepopolato `service: "security-orchestrator"`. Esso si connette al database di sicurezza leggendo i parametri di connessione dall'ambiente, inizializza i client per le varie componenti (verifier JWT, lookup TPM, client OPA, client AI) e crea la cache degli alert Snort con TTL di tre minuti.

Il servizio espone tre handler principali: l'endpoint `/health` per la verifica dello stato di salute, l'endpoint `POST /api/v1/opa/logs` dedicato alla ricezione e memorizzazione in append dei log decisionali compressi provenienti da OPA all'interno del file `opa_decision.jsonl` (che gestisce anche la decompressione gzip), e l'handler principale di valutazione montato su `/api/v1/evaluate` e come catch-all radice.

In parallelo al server HTTP, viene avviato un listener TCP sulla porta 9000 (configurabile tramite la variabile `SNORT_ALERTS_TCP_PORT`) tramite il modulo `snortlistener`. Questo listener riceve gli alert JSON line-delimited trasmessi in tempo reale dai parser Snort e li popola nella `SnortCache`, indicizzata per indirizzo IP sorgente con tre campi distinti: `AlertEdge` (coda 1), `AlertInternal` (coda 2) e `AlertMid` (coda 3).

L'handler di valutazione, costruito tramite la funzione `BuildEvaluateHandler` definita in `handler.go`, esegue una sequenza strutturata di operazioni per ogni richiesta ricevuta:

- Ricostruisce il percorso originale della richiesta consultando gli header `X-Original-Uri`, `X-Authz-Request-Path` o il path della richiesta corrente, ed estrae il metodo HTTP. L'indirizzo IP del client viene ricavato dall'header `x-envoy-external-address` (prodotto da Envoy con `use_remote_address: true` e non falsificabile), con fallback all'ultimo elemento di `X-Forwarded-For`.
- Verifica la presenza di un Bearer token nell'header `Authorization`. Se il token è assente, esegue comunque l'analisi di conformità del dispositivo (Shadow Mode) per tracciamento, costruisce un input base e interroga OPA per verificare se la risorsa è pubblica.
- Se il token è presente, ne valida l'integrità e la firma crittografica interrogando il componente `jwtpkg.Verifier` che scarica dinamicamente le chiavi pubbliche dall'endpoint JWKS dell'identity service.

- Analizza il certificato mTLS client eventualmente inoltrato da Envoy nell'header `X-Forwarded-Client-Cert` tramite il modulo `cert`.
- Interroga in sola lettura la collezione `device_fingerprints` del database di sicurezza tramite il modulo `tpm` per verificare la presenza di una registrazione hardware associata al dispositivo dell'utente.
- Conduce una verifica di conformità del dispositivo in modalità trasparente (Shadow Mode) tramite la funzione `evaluateStrictDeviceFingerprinting`: confronta CN e OU del certificato con i dati del database, controlla gli alert Snort attivi nella cache per l'IP del client (e per l'IP di Envoy, per gli alert di tipo `mid` sul canale `ext_authz`), e registra ogni discrepanza senza bloccare il flusso operativo.
- Costruisce l'evento AI con un vettore di feature che include la consistenza JA3, la presenza di alert Snort per ciascuna delle tre code, il metodo HTTP, il ruolo e il livello di clearance dell'utente e il tier di autenticazione. Invia l'evento al client del sistema di intelligenza artificiale (`aiscorer.Client`) tramite una chiamata HTTP all'endpoint `/infer` del microservizio `ai-inference` per ottenere lo score di anomalia. In caso di timeout o errore di rete il client restituisce un punteggio di fallback `0.99` con confidenza bassa. Se la decisione OPA è positiva, l'evento viene committato nel modello tramite una chiamata asincrona all'endpoint `/update` (meccanismo anti-poisoning).
- Compone il payload strutturato `opa.Input` includendo il metodo, il percorso, i claim del token, i flag di presenza del certificato e del TPM, i metadati geografici della risorsa (`zone_id`), il punteggio e la confidenza dell'AI, e il contesto temporale e di rete (indirizzo IP del mittente, ora del giorno, giorno della settimana e anzianità della sessione in secondi).
- Invia la richiesta al motore OPA. In caso di errore del motore di policy, l'orchestratore applica una logica fail-closed restituendo uno status `403`; in caso di esito positivo, risponde a Envoy con uno status `200` e inserisce gli header `x-current-user` e `x-envoy-external-address` per propagare l'identità dell'utente verificato e il suo IP al backend; in caso di diniego, restituisce uno status `403`.

4.5 Open Policy Agent e la policy in Rego

Il motore Open Policy Agent valuta la policy definita in `infra/opa/policy.rego`. La policy dichiara l'uso della sintassi recente tramite `import rego.v1` all'interno del package `envoy.authz`.

La policy adotta un approccio fail-closed configurando come prima istruzione la direttiva `default allow := false`. L'architettura della policy è organizzata in quattro sezioni principali.

Sezione 1 — Rotte pubbliche. Un primo insieme di percorsi tecnici (`/health`, `/well-known/jwks.json`, `/favicon.ico`) vengono autorizzati incondizionatamente sen-

za alcuna valutazione AI. Un secondo insieme di percorsi pubblici (`public_paths`), che include le rotte di autenticazione, le pagine UI e gli endpoint WebAuthn anonimi, viene autorizzato condizionalmente solo se il punteggio AI netto (score meno uno sconto di 0.2 applicato ai percorsi pubblici) rimane inferiore alla soglia di 0.6. Tutti i percorsi con prefisso `/static/` sono autorizzati senza restrizioni.

Sezione 2 — Matrice di sicurezza. Per le risorse protette, l'autorizzazione si basa su una struttura dati `matrice_sicurezza` che associa a ciascuna rotta base e metodo HTTP tre parametri: `ruoli_ammessi` (lista dei ruoli abilitati), `benefici` (valore numerico che riduce il rischio effettivo) e `rischio_accettato` (soglia massima di rischio netto). La struttura copre cinque categorie di risorse: personale (`/api/v1/personnel`), documenti (`/api/v1/documents`), materiali nucleari (`/api/v1/nuclear-materials`), parametri del reattore (`/api/v1/reactor-parameters`) e l'endpoint di cancellazione sanitizzata tramite Trusted Guard (`/api/v1/trusted-guard/sanitized-delete-personnel`). Sono incluse anche le rotte di enrollment WebAuthn protette (`/api/v1/auth/register/begin` e `/api/v1/auth/register/finish`).

Sezione 3 — Risoluzione dinamica della rotta. La policy implementa un meccanismo di risoluzione della rotta base (`rotta_base`) che gestisce le sottorotte con parametri di percorso: una richiesta verso `/api/v1/documents/abc123` viene ricondotta alla rotta base `/api/v1/documents` applicando un confronto per prefisso. In presenza di più rotte compatibili, viene selezionata quella con il prefisso più lungo (match più specifico).

Sezione 4 — Regola di autorizzazione. La regola principale concede l'accesso se sono soddisfatte due condizioni: il ruolo dell'utente (`input.claims.role`) è presente nella lista `ruoli_ammessi` per la combinazione rotta/metodo, e il rischio netto (score AI meno i `benefici` della rotta) è inferiore al `rischio_accettato`. Questa formulazione integra nativamente il controllo basato sui ruoli (RBAC) con la valutazione del rischio contestuale fornita dall'AI, senza separare i due meccanismi.

Rispetto alla versione precedente, la policy ha abbandonato i concetti di Tier di dispositivo e clearance documentale come requisiti espliciti, sostituendoli con un modello unificato in cui il livello di autenticazione (presenza del certificato, verifica TPM) contribuisce implicitamente allo score AI come feature del modello, mentre il controllo degli accessi formale è ridotto al solo ruolo. Il rischio AI incorpora tutte le informazioni contestuali: ora del giorno, anzianità della sessione, alert Snort, metodo HTTP e sensibilità della risorsa.

4.6 Il servizio Identity

Il servizio Identity gestisce l'autenticazione a due fattori degli utenti e l'enrollment dei dispositivi hardware. Scritto in Go, il punto di avvio risiede in `cmd/identity/main.go`.

All'avvio, il servizio inizializza un logger JSON strutturato che scrive nel file `identity_events.json` inserendo il tag di servizio `"identity-service"`, si connette al database di sicurezza tramite il modulo `config.LoadConfig()`, istanzia il gestore JWT (`crypto.JWTManager`) basato su chiavi RSA asimmetriche e configura il client SMTP per l'invio degli OTP.

In background, un ticker ogni cinque minuti esegue la pulizia delle sessioni di rate limit scadute.

Il routing, descritto nel file `routes.go`, definisce la struttura degli endpoint del servizio: l'endpoint `/health` per la diagnostica, l'endpoint `/.well-known/jwks.json` per l'esposizione delle chiavi pubbliche RSA, le rotte UI per le pagine di login e registrazione, le API funzionali per l'accesso (`/api/v1/auth/register`, `/api/v1/auth/login` e `/api/v1/auth/verify-otp`) e gli endpoint delle cerimonie WebAuthn (`/api/v1/auth/register/begin`, `/api/v1/auth/register/finish`, `/api/v1/auth/login/begin`, `/api/v1/auth/login/finish`).

Il processo di autenticazione iniziale è implementato nell'handler `Login` contenuto in `login.go`:

- Riceve le credenziali dell'utente, estrae l'indirizzo IP del client dall'header `x-envoy-external-address` e verifica il rate limit per quell'indirizzo. Se il limite è superato, la richiesta viene rifiutata con `429 Too Many Requests`.
- Analizza il certificato mTLS client dall'header `X-Forwarded-Client-Cert` tramite la funzione `extractCertFields`. Se il certificato è assente e il ruolo dell'utente nel database non è `guest`, la richiesta viene bloccata immediatamente con `403 Forbidden`: i ruoli non-guest richiedono sempre il certificato mTLS per autenticarsi. Se il certificato è presente, il Common Name deve coincidere con lo username (case-insensitive) e l'Organizational Unit deve corrispondere al ruolo dell'utente nel database. Qualora si rilevi una divergenza in uno dei due campi, la richiesta viene bloccata e il fallimento viene registrato nel contatore di rate limit.
- Effettua la verifica della password con `Argon2id`. Se lo username non corrisponde ad alcun utente registrato, il sistema esegue un confronto fittizio con un hash di prova per bilanciare i tempi di computazione e neutralizzare i tentativi di enumerazione basati sulla misurazione delle latenze di risposta (timing attack).
- A seguito della corretta validazione della password, genera un codice OTP a sei cifre, calcola l'hash HMAC-SHA256 dell'OTP usando il session token come chiave, crea una sessione OTP nel database con TTL di cinque minuti e limite massimo di tre tentativi, e avvia in background (goroutine con `recover`) l'invio dell'OTP tramite SMTP all'indirizzo email dell'utente. L'invio asincrono non blocca la risposta HTTP: la sessione è già salvata in DB e l'utente può riprovare in caso di errore di consegna.

Il completamento dell'autenticazione avviene tramite l'handler `VerifyOTP` descritto in `verify_otp.go`:

- Riceve il codice di sessione e il valore dell'OTP.
- Esegue un'operazione atomica sul database (`ConsumeAttempt`) che incrementa il contatore dei tentativi filtrando contemporaneamente sui criteri `{token, attempts < MAX}`, garantendo la protezione contro attacchi a condizioni di concorrenza (race condition tra verifica del limite e incremento).

- Verifica l'OTP confrontandolo con l'hash HMAC-SHA256 memorizzato, usando il session token come chiave di autenticazione. Se la verifica fallisce, comunica il numero di tentativi rimanenti.
- Se l'OTP corrisponde, cancella la sessione provvisoria, recupera i dettagli dell'utente, interroga il database per recuperare il più recente dispositivo WebAuthn registrato (per ottenere il `device_id` da incorporare nel JWT) ed emette il token JWT firmato con algoritmo RS256, all'interno del quale vengono incorporati l'identificativo dell'utente, il ruolo, il livello di clearance, l'impronta JA3 e il `device_id`, con durata configurata tramite la costante `AccessTokenTTL`.
- Aggiorna in background i dettagli dell'ultimo accesso memorizzando l'indirizzo IP, il timestamp e l'impronta JA3.

4.7 Il servizio Business Logic

Il servizio Business Logic implementa le API REST dedicate alla gestione delle risorse della centrale. Sviluppato in Go, il servizio configura un logger JSON strutturato con il tag "business-logic" per ogni evento registrato e istanzia i repository MongoDB per ciascun dominio applicativo.

Il server configura le rotte applicative sfruttando le funzionalità introdotte in Go 1.22 per la definizione dei parametri di percorso all'interno del `ServeMux`. Le rotte sono organizzate in tre domini semantici:

- *Dominio gestione* (RBAC gerarchico standard): il servizio espone le operazioni di lettura e creazione per la risorsa `personnel` (accessibile da operator, manager e admin) e le operazioni di lettura, creazione e cancellazione per le risorse `documents` e `nuclear-materials` (accessibili da manager e admin).
- *Dominio reattore e nucleo* (Bell-LaPadula rigoroso): le operazioni di lettura, creazione e cancellazione per la risorsa `reactor-parameters` sono riservate al solo ruolo admin, implementando il principio di no-read-up per gli operatori e i manager.
- *Trusted Guard / Gateway di sanitizzazione*: la rotta `POST /api/v1/trusted-guard/sanitized-d` consente all'admin di cancellare un record di personale (operazione che violerebbe formalmente Bell-LaPadula per write-down) attraverso un gateway di sanitizzazione. L'handler `TrustedGuardDeletePersonnel` risponde immediatamente con `202 Accepted` e delega l'effettiva cancellazione a una goroutine asincrona che introduce un jitter temporale casuale tra due e sei ore prima di eseguire la `DELETE` sul repository, usando un contesto indipendente dalla richiesta originale. Ogni fase è tracciata tramite `log_action`.

Il middleware di logging `log_action` intercetta ogni transazione: estrae dall'header `X-Current-User` l'identità dell'utente autenticata (iniettata da Envoy dopo la validazione del Security Orchestrator) e dall'header `X-Request-ID` il tracciante di correlazione, producendo una riga di log JSON strutturata con il nome dell'azione, il target, lo stato e l'eventuale errore.

Per garantire la protezione contro la manomissione diretta dei record nella base dati, ciascun repository applicativo implementa una convalida crittografica tramite la funzione `computeDataIntegrityHash`. All'atto dell'inserimento o della modifica di un documento, il repository calcola un hash SHA-256 concatenando i campi informativi del record separandoli con il carattere pipe. L'hash risultante viene memorizzato all'interno del campo `data_integrity_hash` dello stesso documento. Ad ogni successiva operazione di lettura, il repository ricalcola l'hash confrontandolo con quello memorizzato: un'eventuale divergenza blocca l'esposizione del record e restituisce un errore di violazione dell'integrità.

4.8 I database MongoDB

Il sistema impiega due istanze MongoDB separate per isolare le componenti di sicurezza dai dati applicativi della centrale.

Il Business DB gestisce i dati applicativi ed è configurato abilitando l'autenticazione interna delle connessioni. Lo script di configurazione crea quattro utenze con privilegi rigidamente circoscritti: l'utenza `envoy_service` dispone dei diritti di lettura e scrittura per le normali operazioni delle API, l'utenza `seed_service` ha accesso completo per la popolazione iniziale delle collezioni, l'utenza `splunk_reader` ha permessi di sola lettura per la raccolta dei log e l'utenza `pdp_reader` dispone di accesso in sola lettura per consentire le verifiche del motore di policy.

Tutte le collezioni del database sono protette da validatori JSON Schema che vincolano i tipi di dato, la presenza di attributi obbligatori e il formato degli identificatori tramite espressioni regolari. Inoltre, i documenti del personale e delle zone integrano i sottomenti `ztna_metadata` e `ztna_policy` per memorizzare in modo sicuro i punteggi di fiducia, le regole di ammissione e i parametri di conformità richiesti per l'accesso.

Il Security DB ha una configurazione finalizzata alla gestione degli accessi: memorizza le credenziali Argon2id degli utenti nella collezione `identity_users` (con campo `has_tpm` per il flag WebAuthn e struttura `last_login_info` con IP, timestamp e impronta JA3) e i dati dei dispositivi hardware accreditati nella collezione `device_fingerprints`, rimanendo inaccessibile a servizi esterni al piano di autenticazione e decisione.

4.9 Il seeder del Business DB

Il popolamento iniziale del Business DB è affidato a un'applicazione Go autonoma posizionata in `tools/seedr/`. L'applicazione si connette al database di produzione ed esegue l'inserimento dei record seguendo l'ordine imposto dalle dipendenze referenziali.

Il processo di inserimento avviene secondo la sequenza logica descritta nell'elenco seguente:

- inserimento delle zone fisiche della centrale;
- inserimento dei record del personale operante;

- associazione dei badge fisici di accesso e dei log di transito storico;
- popolamento dei parametri del reattore, degli ordini di manutenzione, dei documenti tecnici e dei materiali nucleari.

Ciascun inserimento verifica preventivamente la presenza di record per evitare duplicazioni, garantendo l'idempotenza del processo. L'applicazione calcola e popola gli hash crittografici di integrità per ciascun documento, rendendo la base dati sicura e verificabile fin dalla prima attivazione.

4.10 Il forwarder Splunk e l'osservabilità centralizzata

Il monitoraggio centralizzato si avvale di un'istanza Splunk Enterprise combinata con un Universal Forwarder (`splunk-uf`). La configurazione del forwarder è definita nel file `inputs.conf`, il quale monitora nove directory di volume distinte contenenti i log in formato JSON strutturato prodotti da tutti i componenti dell'infrastruttura: il firewall `nftables`, le tre istanze Snort (esterna, interna e intermedia), Envoy, il business-logic, MongoDB, l'identity service, il Security Orchestrator e OPA.

L'inoltro dei dati all'istanza Splunk avviene tramite la configurazione del file `outputs.conf` puntando alla porta standard del protocollo di indicizzazione. La correlazione end-to-end è realizzata propagando l'identificativo `X-Request-ID` generato da Envoy a tutti i componenti coinvolti: una singola ricerca in linguaggio SPL sul valore dell'identificatore consente di ricostruire la sequenza temporale di tutti gli eventi associati a una determinata richiesta, attraversando i log di Envoy, del Security Orchestrator, dell'OPA e del servizio business applicativo.

4.11 La suite di test e validazione automatizzata

L'affidabilità e l'efficacia delle misure di sicurezza implementate sono verificate in modo continuo attraverso una suite integrata di test e strumenti di validazione, coordinati dallo script principale `tests/e2e/run_all.sh`. La suite esegue sei pillar di test in sequenza e compila automaticamente il file di riepilogo `tests/e2e/REPORT.md` con una tabella riassuntiva e l'output dettagliato di ciascun pillar.

I sei pillar della suite coprono i seguenti aspetti:

- *auth*: verifica il flusso di autenticazione completo (login con credenziali, ricezione OTP, verifica OTP e ottenimento del JWT).
- *pep*: verifica che il Policy Enforcement Point (Envoy + `ext_authz`) blocchi le richieste non autorizzate e consenta quelle legittime.
- *nftables*: valida il firewall attraverso sei sub-test automatizzati che verificano: l'accettazione del traffico legittimo verso Envoy, la corretta composizione della chain output con policy DROP e allow-list limitata alle porte dei servizi Go, la presenza

del file JSONL prodotto dal parser Go con record **action** validi, e la presenza della regola di rate-limiting anti-SYN flood nella chain input.

Una componente separata di test di simulazione è presente in **tests/clients/**, dove client scritti in Go e Python possono generare scansioni di porte (SYN su porte consecutive) e handshake TLS malformati per sollecitare i meccanismi di rilevamento di Snort, e in **tests/opa/** per gli unit test della policy Rego.

5 Il modello Graphagate: Temporal Graph Network per la Zero Trust Architecture

5.1 Introduzione

Nel contesto di una Zero Trust Architecture (ZTA), il processo di autorizzazione continua richiede meccanismi dinamici per rilevare deviazioni dal comportamento abituale e segnali di compromissione. Il modello integrato con l'Orchestrator OPA è basato su una *Temporal Graph Network* (TGN), progettata specificamente per il *self-supervised anomaly detection* su stream di traffico continuo.

Ogni accesso (una richiesta IP/device verso una risorsa) rappresenta un arco temporale all'interno di un grafo in evoluzione. Il modello mantiene uno stato ricorrente per ciascuna entità (utente o risorsa) e analizza i vicini temporali recenti per assegnare uno score di anomalia in tempo reale, consentendo di bloccare le richieste ostili ed esfiltranti.

5.2 Il modello base: Temporal Graph Network (v3)

5.2.1 Architettura del sistema

L'architettura del modello combina la memoria ricorrente tipica dei TGN (Rossi et al., 2020) con un gestore del vicinato ottimizzato (bounded in RAM) e uno *scorer* a due teste che permette di analizzare le anomalie su due fronti: contestuale/policy e strutturale.

Componenti principali

- *TGNMemory*: struttura basata su GRU che aggiorna lo stato storico di ciascun nodo con i messaggi degli eventi in ingresso. La dimensione è impostata a 256 per gestire le impronte comportamentali.
- *Hashed Identity*: identità dei nodi mappate tramite funzioni di hash (`hash(URI) % hash_buckets`) per garantire totale *induttività* e scalabilità in presenza di entità mai viste prima.
- *MessageNeighborLoader*: buffer in RAM che memorizza gli ultimi vicini temporali di ciascun nodo (es. `neighbor_size=30`). Lavora con lookup in tempo $O(1)$ su matrici preallocate ed elimina la necessità di lenti database a grafo.
- *GraphAttentionEmbedding*: rete multi-hop con *TransformerConv* per calcolare l'embedding attendendo sui vicini storici, concatenando l'identità alla storia temporale.

- *Static Node Features*: vettore a 16 dimensioni associato a ciascun nodo, che codifica attributi ZTA essenziali come *device tier*, *clearance* e *trust score*, oltre a metriche estese introdotte nello schema v3 quali il *resource risk* (per innalzare la sensibilità su asset critici) e il *source internal/external flag* (per distinguere reti RFC1918 da IP pubblici; ablatabile e *disattivato di default* nella configurazione corrente, poiché il roaming esterno benigno può mascherare il furto di credenziali — la coppia *external*→*device* che esporrebbe l'attaccante), riportati di seguito.
- *History Features*: per ogni arco valutato si calcolano tre contatori causali e *benign-gated* (aggiornati solo su traffico approvato). A questi si aggiunge una seconda tripletta identica per le coppie *aux_src*→*dst* (es. per validare la *device habituality*), portando la dimensione a 6. Misurano quanto è abituale quell'accesso: una coppia mai vista da una sorgente attiva è la *novelty cue* del movimento laterale, che, combinata con la memoria temporale, porta il maggior contributo (ablation multi-seed a 3 seed: +0.163 AUC, il segnale dominante del laterale).

$$[\log(1+\text{pair_count}), \log(1+\text{src_count}), \text{pair_count}/(\text{src_count}+1)]$$

- *Dual Head Scorer*: lo score di un arco è la *somma* di due logit complementari, $\text{logit} = \text{feat_logit} + \text{struct_logit}$.
 - *Feature head* (LinkPredictor, MLP a 3 strati): consuma gli embedding dei due endpoint, il messaggio dell'evento, gli attributi statici con hashed-identity, le codifiche di recency (Δt coppia, Δt sorgente) e le *history features*. Cattura anomalie *contestuali* e di *policy* e veicola la *novelty cue* del laterale.
 - *Structural head*: similarità coseno (scalata da una temperatura apprendibile) delle proiezioni non-lineari degli embedding. È *marginale* sul laterale (ablation a 3 seed: +0.007 AUC), poiché su questo stream il segnale è già portato dalle *history features* e dalla memoria temporale; è comunque *mantenuta* come segnale complementare a basso costo (l'ottimizzatore le assegna una temperatura non banale, $\text{struct_scale} \approx 4.7$, dunque è attiva ma ridondante). Il suo contributo è plausibilmente sottostimato dal task sintetico - dove il laterale coincide quasi per costruzione con la *pair-novelty* già catturata dalle *history features* - e atteso più rilevante su topologie reali o in regime di *cold-start/poisoning* dei contatori, ma ciò resta *non verificato*.
- *Kill-chain Precursor* (prior di serving, non addestrato): il movimento laterale segue tipicamente un alert di recon (Snort / anomalia rilevata) sulla *stessa* entità, ma il gate predict-then-update scarta quel precursore dalla memoria TGN. Si mantiene quindi uno stato per-entità **recent_alert** e si applica un fattore *moltiplicativo* allo score, $1 + \max_boost \cdot 0.5^{\Delta t / \text{half_life}}$ (default $\max_boost=2$, $\text{half_life}=600\text{s}$). Moltiplicativo per costruzione: alza uno score già sospetto sopra soglia senza trasformare in falsi positivi gli score benigni ≈ 0 . Il suo contributo è però *marginale* (ablation multi-seed a 3 seed: +0.013 AUC), comparabile a quello della testa strutturale. Non costituisce un input addestrato (il training solo-benigno lo renderebbe una feature morta).

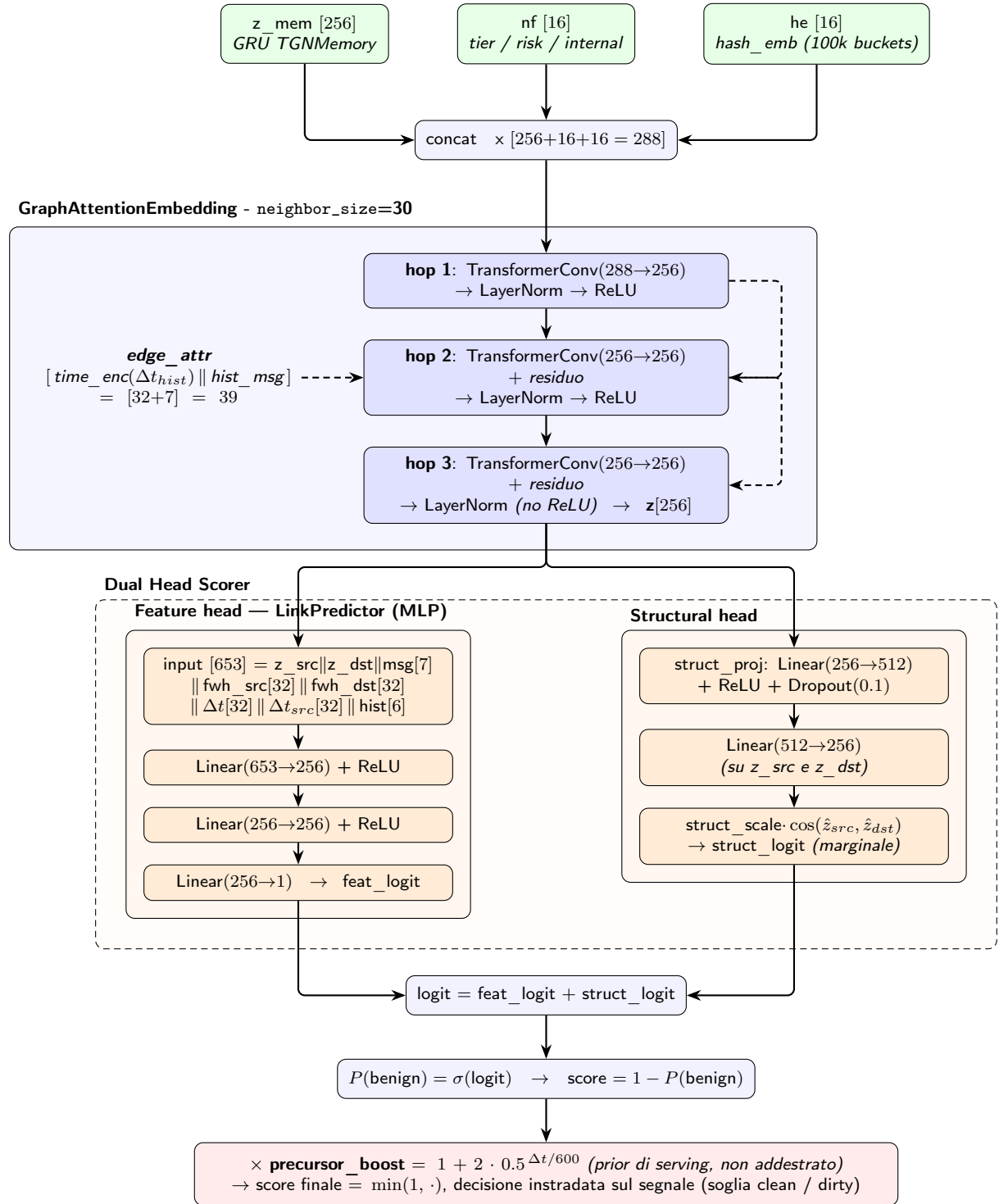


Figura 5.1: Schema dei layer del modello: i tre hop di *GraphAttentionEmbedding* e le due teste (*Feature* e *Structural*) del *Dual Head Scorer*.

5.2.2 Fase di addestramento e calibrazione

Il modello viene addestrato con un approccio *self-supervised* unicamente su traffico benigno, senza mai essere esposto a etichette di anomalia (unsupervised anomaly detection). Lo stream temporale sintetico generato viene suddiviso cronologicamente in tre subset: addestramento (70%), validazione (10%) e test (20%).

L'obiettivo di apprendimento per la rete è il *link prediction*: il TGN è istruito per massimizzare lo score assegnato ad eventi realmente accaduti nella baseline benigna e minimizzare contemporaneamente lo score associato ad eventi artificialmente manipolati e generati a runtime (*negative sampling*).

Negative sampling e funzione di loss

L'ottimizzazione (AdamW) somma tre termini self-supervised. Il dominante è una loss di ranking *InfoNCE* (allineata all'Average Precision): per ogni positivo benigno la sorgente è accoppiata a $K=5$ destinazioni *uniformemente casuali* e si impone che la dst-vera ottenga il logit più benigno *dato lo storico della sorgente* — la rete apprende così $P(\text{dst} \mid \text{src}, \text{storia})$, assegnando score elevato agli accessi a bassa verosimiglianza (movimento laterale, violazioni di policy). La affiancano due ancoraggi BCE: un *positive anchor* che spinge i positivi benigni verso “benigno” fissandone la scala assoluta (l'InfoNCE fissa solo l'ordine relativo, e senza questo la soglia calibrata sull'FPR perderebbe significato), e dei *contextual negatives* che, sommando rumore gaussiano ($\sigma=0.5$) al messaggio dell'evento, addestrano la *Feature Head* a marcarlo anomalo — meccanismo distinto dalla corruzione discreta usata in valutazione, così il modello non vede il test.

Calibrazione della soglia (cost-sensitive instradata sul segnale)

Il TGN non emette una classificazione netta ma uno score continuo confrontato con una soglia, calibrata via *replay* sul 10% di validazione held-out (lo split di test non è mai toccato in calibrazione). Poiché a serving la classe vera è ignota ma il *segnale di edge* (JA3/Snort/sensori) è osservabile, si calibrano e si instradano due soglie:

- *Soglia signal-dirty* (eventi il cui segnale già scatta): il classico quantile al Target FPR dell'1% sugli score benigni. Questi eventi sono già intercettati dalla cheap rule baseline, quindi si mantiene la soglia conservativa.
- *Soglia signal-clean* (dove il laterale è indistinguibile dal benigno se non per il pattern temporale): soglia *cost-sensitive* che minimizza $\text{cost_ratio} \cdot \text{FN} + \text{FP}$ (default $\text{cost_ratio}=20$: un laterale mancato costa molto più di un falso allarme ri-sfidabile dall'orchestrator). Un *cap* esplicito sull'FPR del flusso clean ($\text{clean_fpr_cap}=0.05$) impedisce che la spinta sul recall faccia esplodere i falsi positivi — necessario perché, pur con un'AUC laterale alta, il trade-off recall/FPR sul flusso clean resta ripido.

A inferenza la decisione è instradata per-evento: `signal_dirty` $\Rightarrow$ soglia dirty, altrimenti soglia clean (cost-sensitive). Entrambe le soglie sono persistite ed esposte da `serve_api` (anche su `/health`). OPA resta il decisore finale allow/deny.

5.2.3 Tipologie di anomalie rilevate in produzione

In fase di valutazione o in ambiente live, l'architettura rileva 3 macro-categorie di attacco, dimostrando vantaggi misurabili rispetto a baseline più semplici (rule-based, Isolation Forest, One-Class SVM e GNN non temporale; cfr. Sezione 5.2.4):

1. *Policy Anomaly*: l'entità agisce al di fuori del proprio ruolo, clearance o device tier. Intercettata agevolmente dalla Feature head grazie alle feature statiche ZTA.
2. *Contextual Anomaly*: la connessione ha caratteristiche insolite o ha innescato sensori IPS. Rilevata dalla Feature head decostruendo le *edge features*.
3. *Lateral Movement*: l'accesso è formalmente autorizzato a livello di regole OPA, ma *non abituale*. Avviene tipicamente nel "pivoting". Essendo indistinguibile sintatticamente dal traffico benigno, la classificazione ricade quasi del tutto sull'architettura implementata.

5.2.4 Valutazione sperimentale e confronto con le baseline

La valutazione è condotta sullo split di test (20% finale dello stream sintetico) tramite replay per-evento attraverso l'esatto codice di serving (`num_events`=200 000, di cui il 20% finale = 40 000 eventi di test), su **3 seed** [42, 7, 123] (media $\pm$ dev.std). Per quantificare il contributo *effettivo* della componente temporale, il TGN è confrontato con tre baseline non supervisionate addestrate sullo *stesso* traffico benigno, con identico split cronologico, identici segnali tabellari (contatori di storia + prior precursore), soglia calibrata all'1% di FPR e protocollo di valutazione (codice in `tests/baselines/`); una quarta baseline *supervisionata* (XGBoost) è riportata solo come upper-bound di riferimento:

- *Isolation Forest*: modello non relazionale addestrato sulle sole feature statiche per-evento (nessuna struttura di grafo), privo di informazione strutturale (lateral AUC 0.645 ± 0.025).
- *One-Class SVM*: controparte kernel non relazionale dell'Isolation Forest (sole feature per-evento). Pur con AUC aggregata competitiva (0.801 ± 0.018), sul *lateral* resta a 0.599 ± 0.002 , confermando che il segnale temporale non è catturabile dalle sole feature statiche.
- *GNN non temporale*: ablation del TGN su grafo statico aggregato, con InfoNCE su destinazioni casuali più il rumore contestuale e le stesse *history features*, ma priva di memoria ricorrente e vicinato temporale (lateral AUC 0.451 ± 0.021 , $\approx$ caso).
- *XGBoost* (*supervisionato*, fuori categoria): addestrato su dataset etichettato (anch'esso generato artificialmente), raggiunge lateral AUC 0.929 ± 0.001 . Non è un confronto equo col paradigma non supervisionato del TGN; è incluso solo per delimitare l'upper-bound raggiungibile avendo accesso alle label (assenti per costruzione nel nostro setting).

I risultati del confronto sono riportati nella Tabella 5.1.

Metrica (test set)	TGN	GNN non temp.	One-Class SVM	Isol. Forest
AUC aggregata	0.965 $\pm$ 0.007	0.555 $\pm$ 0.037	0.801 $\pm$ 0.018	0.763 $\pm$ 0.014
AP aggregata	0.922 $\pm$ 0.014	0.560 $\pm$ 0.021	0.739 $\pm$ 0.026	0.575 $\pm$ 0.016
Lateral — AUC	0.913 $\pm$ 0.014	0.451 $\pm$ 0.021	0.599 $\pm$ 0.002	0.645 $\pm$ 0.025
Lateral — Recall @1%FPR	0.166 $\pm$ 0.009	0.125 $\pm$ 0.006	0.062 $\pm$ 0.016	0.009 $\pm$ 0.002
Recall aggregata @1%FPR	0.665 $\pm$ 0.043	0.366 $\pm$ 0.004	0.479 $\pm$ 0.038	0.116 $\pm$ 0.020

Tabella 5.1: Confronto sul test set sintetico tra TGN e baseline non relazionali (3 seed [42,7,123], soglia globale all'1% FPR); il riferimento *supervisionato* XGBoost (upper-bound fuori categoria) non è incluso qui ed è discusso nel testo e nel quadro riepilogativo (Tab. 5.9).

Cost-sensitive routing

Per convertire la forte capacità di ranking del TGN (Tab. 5.1) in recall operativo, la decisione è instradata sul segnale con la soglia cost-sensitive calibrata in Sez. 5.2.2. Sul test sintetico (3 seed, media) ciò più che raddoppia il recall laterale rispetto alla soglia globale all'1% FPR, al prezzo di un modesto aumento dell'FPR benigno; i valori per-classe e instradati sono riportati nella riga v3 di Tab. 5.3.

5.3 L'evoluzione: il nodo *Configuration* (schema v4)

5.3.1 Motivazione

Il modello descritto nella Sezione 5.2 (schema v3) modella ogni richiesta come una catena causale a tre archi su quattro ruoli di nodo, $source \rightarrow device \rightarrow user \rightarrow resource$. In quello schema il *fingerprint* TLS del client (JA3) è soltanto un *bit di validità* (`features[0]`): segnala se l'handshake è regolare, ma non è un'*identità*. Questo lascia scoperti due vettori d'attacco centrali nella ZTA:

- *Movimento laterale con tooling nuovo*: un dispositivo già noto, compromesso, inizia a parlare con un client/tool mai osservato su quella macchina. La validità TLS resta intatta (il tool usa una libreria regolare), ma il *fingerprint* è nuovo.
- *Furto di credenziali*: un attaccante che riusa le credenziali della vittima da un proprio client presenta un JA3 *diverso* da quello abituale dell'utente, pur con ruolo/clearance leciti (l'attaccante eredita i permessi della vittima).

Entrambi sono *signal-clean* e *policy-clean*: indistinguibili sintatticamente dal traffico benigno se non per l'*identità della configurazione*. Da qui la promozione della configurazione del client a quinto nodo.

5.3.2 Architettura del nodo *Configuration*

La configurazione è identificata da `conf:<ja3>` quando il *fingerprint* è disponibile, altrimenti dal generico `conf:guest` (sempre presente lato serving, default server-side). Il JA3 mantiene quindi un *doppio ruolo*, ortogonale: resta il bit di validità TLS in `features[0]` (catturato dalla *Feature head*) e diventa un'identità di nodo (catturata dalla memoria temporale e dalle *history features*). Per costruzione la dimensione del messaggio (`msg_dim=7`) e quella delle feature statiche di nodo (`node_feat_dim=16`) restano invariate: il nodo config, come i *source*, non porta attributi statici dedicati ma solo memoria ricorrente, hashed-identity e slot di trust. Lo schema dell'evento passa a `schema_version=4` (i checkpoint v1/v2/v3 sono rifiutati al caricamento).

Topologia: la catena causale a 5 archi

Il nodo config è *inserito tra source e device* nella catena causale, sostituendo l'arco diretto `source→device` con la coppia `source→config→device`, e aggiungendo un *binding* discriminante `config→user`. Il cammino più lungo è quindi:

$$source \rightarrow config \rightarrow device \rightarrow user \rightarrow resource.$$

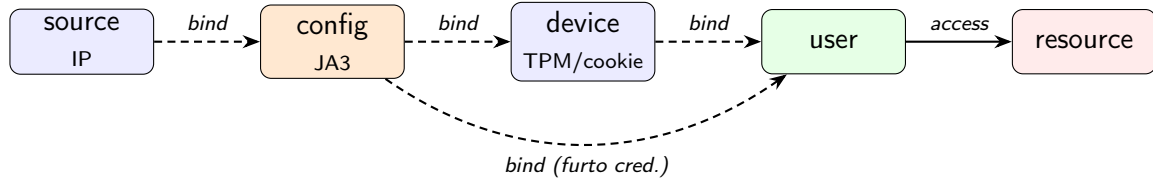


Figura 5.2: Catena causale dello schema v4, con il nodo *config* inserito tra *source* e *device* e il *binding* `config→user`; gli archi tratteggiati sono *binding* a messaggio-zero, il solo `access user→resource` porta il messaggio a 7 dimensioni.

Arco	Tipo	Stato
<code>user → resource</code>	access (msg 7-dim)	esistente
<code>device → user</code>	binding (msg-zero)	esistente
<code>source → config</code>	binding (msg-zero)	nuovo (v4)
<code>config → device</code>	binding (msg-zero)	nuovo (v4)
<code>config → user</code>	binding (msg-zero)	nuovo (v4)

Tabella 5.2: Edge set per richiesta nello schema v4 (config sempre presente lato serving), con l'ordine di commit in memoria identico in training/replay/serving.

L'architettura core (memoria ricorrente, *GraphAttentionEmbedding* multi-hop, *Dual Head Scorer*) è *invariata*: il modello è agnostico al numero di archi per evento, e i nuovi *binding* riusano la stessa primitiva di scoring degli archi esistenti (con *history features* senza ausiliario, come per `device→user`). Il generatore sintetico è esteso coerentemente: ogni

macchina riceve 1–2 configurazioni abituali da un pool condiviso; il movimento laterale presenta con probabilità 0.5 un *tool* (config) mai usato da quel device; il furto di credenziali alloca al client attaccante un JA3 mai visto (`conf:atk-NNN`). Inoltre, per supportare rotte pubbliche (es. `/login`) che non richiedono un JWT e prevenire falsi positivi da cold-start, lo stream inietta regolarmente accessi a tali risorse usando lo slot utente `anonymous` combinato al fallback `dev:guest`, insegnando al modello la normalità del traffico non autenticato.

5.3.3 Risultati: il contributo del nodo config

La valutazione segue lo stesso protocollo della Sezione 5.2 (replay per-evento attraverso l'esatto codice di serving, split cronologico 70/10/20, soglia cost-sensitive instradata). Si riportano due confronti complementari: (i) il modello v3 vs v4 (*keying* per-cookie) all'*operating point* instradato (**3 seed** [42,7,123], media $\pm$ dev.std, 200 000 eventi / 15 epoche), e (ii) la *validazione mirata* del nodo config — un'ablazione controllata *a parità di dati* su seed multipli.

Confronto v3 $\rightarrow$ v4 (*keying* per-cookie)

La Tabella 5.3 confronta l'artefatto (*keying* per-cookie, coerente con la riga TGN della Tab. 5.1) prima e dopo l'introduzione del nodo config, al medesimo *operating point* (decisione cost-sensitive instradata sul segnale), su 3 seed. Il guadagno *robusto* è sul recall operativo del movimento laterale, che cresce oltre la banda di rumore multi-seed — coerentemente con la validazione *theft-rich* (Tab. 5.4) — mentre anche l'AUC laterale migliora in modo significativo; le metriche aggregate restano invece piatte entro il rumore. Il *trade-off*, leggibile in tabella, è un FPR benigno instradato più alto e un'AP aggregata lievemente inferiore: il nodo config non domina uniformemente, ma *sposta* l'*operating point* verso il laterale. È il comportamento atteso, poiché *config* $\rightarrow$ *device* cattura il “tool nuovo su device noto” e *config* $\rightarrow$ *user* il “client che quell'utente non usa abitualmente”, cioè i *tell* del laterale e del furto di credenziali.

Validazione mirata: il furto di credenziali

Per quantificare il contributo specifico del binding *config* $\rightarrow$ *user*, è stata eseguita una validazione mirata su uno stream *theft-rich* (slot e tasso d'incidente aumentati solo per questo test), confrontando - a parità di dati, su 3 seed (media $\pm$ dev.std) e con `save=False` (l'artefatto in `public/` non viene toccato) - il modello v4 completo contro la versione priva del nodo di configurazione (il modello ricade sul percorso legacy *source* $\rightarrow$ *device*, cioè $\approx$ v3 sugli stessi dati). I risultati sono in Tabella 5.4.

Sweep architetturale

L'introduzione di un quinto nodo suggerisce di verificare se al modello serva maggiore capacità espressiva. Tre direzioni — un layer nascosto aggiuntivo nell'MLP della *Feature head* (`link_pred_hidden_layers=3`), una memoria ricorrente più ampia (`memory_dim=384`) e

Metrica (test set)	v3 (4 nodi)	v4 (nodo config)	Δ
AUC aggregata	0.965 $\pm$ 0.007	0.964 $\pm$ 0.010	$\approx 0^\dagger$
AP aggregata	0.922 $\pm$ 0.014	0.897 $\pm$ 0.038	-0.025 [†]
Recall aggregata (intradata)	0.758 $\pm$ 0.030	0.829 $\pm$ 0.040	+0.071
Lateral — AUC	0.913 $\pm$ 0.014	0.930 $\pm$ 0.013	+0.017
Lateral — AP	0.522 $\pm$ 0.020	0.575 $\pm$ 0.098	+0.053 [†]
Lateral — Recall (intradata)	0.350 $\pm$ 0.034	0.577 $\pm$ 0.057	+0.227
Lateral — Recall (@1%FPR)	0.166 $\pm$ 0.009	0.363 $\pm$ 0.128	+0.197
Policy — AUC	0.987 $\pm$ 0.007	0.972 $\pm$ 0.015	-0.015
Contextual — AUC	0.994 $\pm$ 0.001	0.991 $\pm$ 0.002	-0.003
FPR benigno (intradato)	0.050 $\pm$ 0.002	0.066 $\pm$ 0.013	+0.016

Tabella 5.3: Confronto v3 vs v4 (*keying* per-cookie) sul medesimo test set sintetico, decisione cost-sensitive intradata (3 seed [42,7,123], media $\pm$ dev.std); [†] marca i Δ entro la banda di rumore multi-seed, quindi non significativi.

Metrica (test, theft-rich)	v4 (nodo config)	$\approx$ v3 (ablazione)	Δ
Furto credenziali — AUC	0.729 $\pm$ 0.088	0.676 $\pm$ 0.036	+0.053
Furto credenziali — Recall@thr	0.314 $\pm$ 0.107	0.206 $\pm$ 0.039	+0.108
Lateral — AUC	0.919 $\pm$ 0.018	0.913 $\pm$ 0.007	+0.006
Lateral — Recall@thr	0.469 $\pm$ 0.088	0.334 $\pm$ 0.014	+0.135
AUC aggregata	0.958 $\pm$ 0.011	0.956 $\pm$ 0.005	+0.002
Recall aggregata (intradata)	0.777 $\pm$ 0.047	0.744 $\pm$ 0.005	+0.033
FPR benigno (cookie-wipe)	0.060 $\pm$ 0.021	0.038 $\pm$ 0.033	+0.022

Tabella 5.4: Validazione mirata del nodo config su stream *theft-rich* (3 seed [42,7,123], `save=False`): il modello v4 completo contro la v3 a parità di dati (percorso legacy *source* $\rightarrow$ *device*, $\approx$ v3).

più teste di attenzione (`gnn_heads=8`) — e la loro combinazione, valutate su 3 seed con `save=False`, non migliorano il movimento laterale rispetto alla baseline v4: il layer MLP aggiuntivo lascia l'AUC invariata ma abbassa il recall, mentre memoria e teste più ampie degradano nettamente le metriche e aumentano la varianza tra seed. Il segnale laterale è dunque *signal-bound*, non *capacity-bound*: l'architettura corrente è già adeguata e resta invariata.

Sintesi

Il nodo *configuration* (v4) alza sostanzialmente il **recall operativo** del movimento laterale e abilita la rilevazione del furto di credenziali (Sez. 5.3.3), senza costi architetturali:

`msg_dim` e `node_feat_dim` restano invariati e lo sweep conferma che la capacità attuale è già adeguata. Come discusso sopra non è un dominio uniforme ma uno *spostamento dell'operating point* verso il laterale, al costo di un FPR benigno più alto. Questo guadagno operativo si *somma* a quello già documentato del TGN sulle baseline non-relazionali (Tab. 5.1), che non modellano la configurazione del client.

5.3.4 Esperimento: granularità dell'identità del dispositivo (nodo `dev:guest`)

Nello schema v4 ogni dispositivo senza TPM è un *nodo distinto*, identificato da un cookie persistente per-macchina (`ck:<id>`) con re-keying su un nodo “freddo” a ogni *cookie-wipe*. In analogia col fallback `conf:guest` (Sez. 5.3), si è valutato se convenga invece *collassare tutti i device non-TPM su un unico nodo condiviso* `dev:guest`: l'identità per-cookie aggiunge potere discriminante (un device mai visto è una novità) o è solo *rumore*, dato che laterale e furto credenziali sono *signal-clean* e portano il segnale sulle edge *config/source/user*? Il comportamento è esposto dal flag `guest_device_fallback` (default *on*, configurazione deployabile): quando attivo, ogni device non-TPM — *incluso quello dell'attaccante* nel furto di credenziali, anch'esso privo di TPM — collassa su `dev:guest` sia in addestramento (instradamento sul medesimo slot) sia a serving, e il *cookie-wipe* è *neutralizzato* (un nodo condiviso non ha un cookie per-macchina da resettare).

Il confronto *keying* per-cookie vs `dev:guest` è condotto su due stream, a parità di dati, 3 `seed` [42,7,123] ($\text{media} \pm \text{dev.std}$) e `save=False`: lo stream *theft-rich* della Sez. 5.3.3 (80k/12ep, slot 400/120, dove furto e wipe ricadono nel test split; Tab. 5.5) e lo stream *standard* (200k/15ep, il protocollo instradato di Tab. 5.3, confronto appaiato per seed; Tab. 5.6).

Metrica (test, theft-rich)	cookie (baseline)	dev:guest	Δ
Lateral — AUC	0.915 ± 0.019	0.911 ± 0.007	-0.004
Lateral — AP	0.583 ± 0.068	0.632 ± 0.017	$+0.049$
Lateral — Recall@thr	0.453 ± 0.060	0.488 ± 0.019	$+0.035$
Furto credenziali — AUC	0.701 ± 0.078	0.804 ± 0.024	$+0.103$
Furto credenziali — Recall@thr	0.244 ± 0.082	0.287 ± 0.062	$+0.043$
AUC aggregata	0.954 ± 0.013	0.961 ± 0.001	$+0.007$
Recall aggregata (instradata)	0.764 ± 0.036	0.787 ± 0.006	$+0.023$
FPR benigno (instradato)	0.045 ± 0.002	0.037 ± 0.004	-0.008
FPR cookie-wipe	0.041 ± 0.013 ($n=651$)	— ($n=0$)	neutralizzato

Tabella 5.5: Esperimento sull'identità del device: *keying* per-cookie vs collasso su `dev:guest` (3 `seed` [42,7,123], stream *theft-rich*, `save=False`); confronto distribuzionale, non appaiato evento-per-evento.

Metrica (test, stream standard)	cookie (baseline)	dev:guest (deployato)	Δ
AUC aggregata	0.979 $\pm$ 0.004	0.965 $\pm$ 0.005	−0.014
AP aggregata	0.955 $\pm$ 0.007	0.926 $\pm$ 0.021	−0.029
Recall aggregata (intradata)	0.851 $\pm$ 0.003	0.811 $\pm$ 0.000	−0.040
Lateral — AUC	0.952 $\pm$ 0.007	0.922 $\pm$ 0.009	−0.030
Lateral — AP	0.749 $\pm$ 0.027	0.652 $\pm$ 0.078	−0.097
Lateral — Recall (intradata)	0.603 $\pm$ 0.006	0.532 $\pm$ 0.028	−0.071
Lateral — Recall (@1%FPR)	0.458 $\pm$ 0.041	0.372 $\pm$ 0.065	−0.086
FPR benigno (intradato)	0.043 $\pm$ 0.008	0.042 $\pm$ 0.007	−0.001

Tabella 5.6: Identità del device sullo *stream standard* (200k/15ep, decisione intradata): *keying* per-cookie vs collasso su `dev:guest` (3 seed [42,7,123], `save=False`, confronto *appaio* per seed). A differenza del regime theft-rich (Tab. 5.5), qui il guest è una *lieve regressione*: lo split di test standard non contiene furto né cookie-wipe.

L'esito è *condizionato allo stream*. Sul **theft-rich** (Tab. 5.5) il collasso su `dev:guest` è un *miglioramento di Pareto*: non degrada il laterale (Δ AUC −0.004, entro il rumore) e migliora nettamente il furto (AUC +0.103). L'identità per-cookie era infatti soprattutto *rumore* — i nodi-cookie sono sparsi e freddi, mentre il segnale d'attacco vive sulle *history features* di *user/config/source*, non sulla novità del device; concentrare il traffico non-TPM su un solo nodo dà *binding* più stabili (FPR benigno −0.008, varianza tra seed molto più bassa) ed elimina l'intera classe di falsi positivi da *cookie-wipe* (n_{wiped} da 651 a 0). Sullo **stream standard** (Tab. 5.6), invece, il test split non contiene furto né wipe (Sez. 5.3.3): quei due vantaggi sono *assenti per costruzione* e affiora solo il costo, una *lieve regressione* sul laterale (AUC −0.030, recall intradata −0.071) e sull'aggregato (AUC −0.014), con FPR piatto. Il guadagno del guest è dunque *condizionato* alla presenza di furto/wipe nello stream.

Sul piano del deployment il flag è lasciato *on* (`dev:guest` di default): in un contesto ZTA reale, dominato da furto di credenziali e dispositivi effimeri, il bilancio è favorevole, al prezzo di un costo che le metriche non catturano — la perdita dell'*attribuzione forense per-macchina*, recuperabile disattivando il flag. Le Tab. 5.3 e 5.1 restano valide come artefatto *per-cookie*, prodotto prima di questa adozione; il confronto onesto col guest è il Δ *appaio* di Tab. 5.6, non i valori assoluti tra run con codice diverso (la lateral AUC per-cookie 0.952 vs lo 0.930 di Tab. 5.3 riflette miglorie di codice successive).

5.4 Deployment e specifiche hardware/software

Le primitive di model-serving (estrazione memoria ed espansione del grafo storico) operano asintoticamente a tempo costante $O(1)$, e il TGN mantiene tutto il suo stato dinamico e ricorrente nativamente in RAM. Da questa scelta discendono due conseguenze di deploy-

ment: il servizio è *single-worker* - lo stato comportamentale per-entità non è replicabile senza frammentarlo (limite discusso in Sez. 5.5) - ed è fondamentale il dimensionamento asincrono.

5.4.1 Piattaforma HW/SW (addestramento e inferenza)

Lo stesso container isolato ospita sia l'addestramento sia l'inferenza. Al fine di calcolare la backward-propagation e smaltire la mole della *Graph Attention*, esso si appoggia alle seguenti specifiche hardware/software:

- container engine nativo agganciato a librerie NVIDIA CUDA 13;
- hardware accelerato su GPU con architettura NVIDIA Blackwell (testato su RTX 5070 Ti). La GPU accelera la *Graph Attention* multi-hop in inferenza e la backward-propagation in addestramento; lo stato ricorrente della `TGNMemory` e i ring-buffer del `MessageNeighborLoader` sono mantenuti con accesso $O(1)$ su tensori preallocati.

Nel momento di chiusura del framework (o tramite specifici endpoint API persistenti), la memoria serializzata di PyTorch (`tgn_checkpoint.pt` e `tgn_stats.json`) viene salvata su disco per evitare cold-start e mantenere lo stato tra diverse esecuzioni del container.

5.4.2 Serving e latenza di inferenza

Il sistema finale eroga un'infrastruttura single-worker fondata su **FastAPI** (gestione I/O asincrono per polling concorrenziale) che elabora gli eventi con una latenza end-to-end (round-trip HTTP client→servizio sulla rete container, catena v4 completa a 5 archi *source→config→device→user→resource*) di $P50 \approx 9.6$ ms ($P99 \approx 10.8$ ms); l'omissione del binding di dispositivo non altera apprezzabilmente né la latenza né la detection in cold-start. Misure su GPU RTX 5070 Ti (VRAM occupata ~ 2.5 GB), tramite test client di streaming live in regime *cold-start* (entità mai viste prima, prefisso `prod_`): un sanity-check d'induttività più che un punto operativo, poiché in cold-start il modello opera ad alto recall ma bassa specificità (lateral recall $\sim 82\%$, specificità benigna $\sim 43\%$ — FPR gonfiato dall'assenza di storico).

5.4.3 Parametri di inferenza forniti dal Security Orchestrator

Il modello non è interrogato direttamente: è il Security Orchestrator a costruire, per ogni richiesta intercettata, l'evento da inviare al microservizio TGN. L'interazione avviene su due endpoint HTTP/JSON distinti, coerenti col gate anti-poisoning della Sez. 5.5:

- `POST /infer` — chiamata in *sola lettura* (`update=False`): restituisce lo score senza mutare la memoria del modello. L'orchestrator lo passa a OPA, che resta il decisore finale *allow/deny*.
- `POST /update` — invocata *solo* se OPA risponde *ALLOW*: *committa* l'evento avanzando lo stato della `TGNMemory` e le code dei vicini temporali. È questo il meccanismo che esclude dallo stato del modello il traffico classificato come ostile.

La risposta è il record `{anomaly_score, is_anomaly, threshold}`. In caso di endpoint non configurato o di errore di rete, il client adotta una politica *fail-suspicious*, restituendo uno score conservativo 0.99 con *confidence* bassa, così che la mancanza del segnale IA non si traduca in un permesso implicito.

Il corpo della richiesta è identico per i due endpoint ed è descritto nella Tabella 5.7. Le chiavi sono *namespaced* per tipo (`src:`, `conf:`, `tpm:`) così che, nel `NodeRegistry` condiviso del modello, un IP non possa mai aliasare uno slot di tipo diverso.

Campo JSON	Obbl.	Significato e costruzione (lato orchestrator)
<code>key_user</code>	sì	Chiave entità utente: lo <code>UserID</code> dei claim, oppure <code>anonymous</code> sulle rotte non autenticate (es. <code>/login</code>).
<code>key_device</code>	no	Chiave device <code>tpm:<id></code> , inviata <i>solo</i> se il TPM è attestato; altrimenti omessa (mai aliasing sull'IP).
<code>key_config</code>	no	Identità di configurazione <code>conf:<ja3></code> dal <i>fingerprint</i> JA3 (header o claim); se non risolvibile, il server adotta il generico <code>conf:guest</code> .
<code>key_source</code>	no	Contesto di rete sorgente <code>src:<ip></code> del client.
<code>key_dst</code>	sì	URI della risorsa, normalizzato per collassare le sottorotte parametriche su una base; deve combaciare con i <code>RESOURCE_URIS</code> usati in addestramento.
<code>timestamp</code>	sì	Istante dell'evento (epoch Unix, intero).
<code>features</code>	sì	Messaggio dell'arco <i>access user→resource</i> : vettore a 7 dimensioni (<code>msg_dim</code>), dettagliato in Tab. 5.8.
<code>user_feat</code>	no	Feature statiche del nodo utente: vettore a 16 dim (<code>node_feat_dim</code>), <i>interamente nullo</i> (contratto di training: ruolo e clearance viaggiano nel messaggio).
<code>device_feat</code>	no	Feature statiche del nodo device: vettore a 16 dim con il solo <code>node_feat[2] = tier</code> (1.0 TPM+cert, 0.5 cert senza TPM, 0.0 nessuno).
<code>dst_feat</code>	no	Feature statiche della risorsa: <i>non inviato</i> (il <i>resource risk</i> è già codificato nel checkpoint addestrato).

Tabella 5.7: Campi del payload JSON inviato dal Security Orchestrator al microservizio TGN (endpoint `/infer` e `/update`).

Il campo `features` è il cuore informativo dell'evento: codifica i segnali Zero-Trust osservabili sull'arco di accesso, nell'ordine fisso riportato nella Tabella 5.8. I primi quattro elementi sono bit di segnale (validità TLS e alert IPS); gli ultimi tre sono attributi della richiesta: il metodo HTTP (0–4) e ruolo/clearance normalizzati in $[0, 1]$.

Il microservizio rifiuta con HTTP 422 ogni payload le cui dimensioni non corrispondano al modello addestrato: `features` deve avere lunghezza esattamente 7 e i vettori di nodo, se presenti, lunghezza 16. Il modello impone inoltre `schema_version=4`: i checkpoint delle versioni precedenti (v1/v2/v3) sono respinti al caricamento.

A titolo di esempio, la richiesta seguente rappresenta un utente *operator* con clearance *CONFIDENTIAL* (`roleVal=1/3`, `clrVal=0.5`) che accede in *GET* (`methodFloat=0`) da un dispositivo attestato via TPM (`device_feat[2]=1.0`), con JA3 coerente (`ja3Float=1.0`) e nessun alert IPS attivo:

Idx	Nome	Valori	Significato
0	ja3Float	{0, 1}	Bit di validità TLS: 1 se il JA3 osservato coincide con quello atteso dai claim, 0 in caso di discordanza.
1	alertEdge	{0, 1}	1 se un alert Snort di <i>edge</i> (perimetro) è attivo per l'IP client o envoy.
2	alertMid	{0, 1}	1 se è attivo un alert Snort <i>mid</i> (intermedio).
3	alertInt	{0, 1}	1 se è attivo un alert Snort <i>internal</i> (interno).
4	methodFloat	0-4	Metodo HTTP: GET= 0, POST= 1, PUT= 2, DELETE= 3, PATCH= 4.
5	roleVal	[0, 1]	Ruolo normalizzato: indice in [guest, operator, manager, admin] diviso per 3.
6	clrVal	[0, 1]	Clearance normalizzata: indice in [PUBLIC, INTERNAL, CONFIDENTIAL, SECRET, TOP_SECRET] diviso per 4.

Tabella 5.8: Composizione del vettore **features** a 7 dimensioni (**msg_dim=7**), nell'ordine esatto con cui l'orchestrator lo costruisce.

```

POST /infer
{
  "key_user":      "u-1042",
  "key_device":    "tpm:dev-1042",
  "key_config":    "conf:769,47-53-5,0-23-65281,29-23-24,0",
  "key_source":    "src:10.0.3.14",
  "key_dst":       "/reactor/telemetry",
  "timestamp":     1750768800,
  "features":      [1.0, 0.0, 0.0, 0.0, 0.0, 0.3333, 0.5],
  "user_feat":     [0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0],
  "device_feat":   [0,0,1.0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
}

```

5.5 Limiti del modello

5.5.1 Assenza di scalabilità orizzontale

Il limite di deployment più strutturale discende direttamente dal design illustrato in Sez. 5.4: lo stato comportamentale per-entità - la memoria ricorrente della **TGNMemory**, i ring-buffer dei vicini temporali del **MessageNeighborLoader** e i contatori espliciti di *history* - risiede interamente in RAM all'interno di un *singolo worker*. Replicare il servizio orizzontalmente (più repliche dietro un *load balancer*) frammenterebbe questo database comportamentale: ogni replica osserverebbe soltanto una frazione dello stream e accumulerebbe una memoria e dei contatori *parziali e incoerenti*, degradando proprio il segnale temporale su cui poggia la rilevazione del movimento laterale. Non esiste inoltre uno stato condiviso a bassa latenza consultabile in tempo $O(1)$ a ogni evento senza reintrodurre quel database a grafo che l'architettura ha esplicitamente eliminato. La scalabilità del sistema

è quindi *verticale* (GPU e RAM più capienti, per sostenere grafi e memorie più ampi) e *asincrona* (l'I/O non bloccante di **FastAPI** assorbe il carico concorrenziale su un singolo processo), non orizzontale. Una distribuzione del traffico tra repliche richiederebbe uno *sharding* coerente per-entità (ogni entità sempre instradata alla medesima replica), con la complessità e i punti di fallimento che ciò comporta — una direzione di lavoro futura, non supportata dall'artefatto corrente.

5.5.2 La sfida del lateral movement

Senza l'ausilio delle etichette (unsupervised), il lateral movement non mostra tracce o indicatori evidenti: è identico (a livello di feature) a un accesso benigno non abituale. Le leve che lo rendono *ordinabile* (lateral AUC 0.913 ± 0.014 , contro ~ 0.45 della GNN statica) sono il vicinato temporale ampio e soprattutto le *history features* esplicite, come quantificato dall'ablation multi-seed in Sez. 5.2; la conversione del ranking in recall operativo passa poi per la soglia cost-sensitive instradata (Sez. 5.2.2, risultati in Sez. 5.2.4). Resta comunque la classe più ostica (AP ~ 0.52), pur nettamente sopra le baseline (cfr. Sez. 5.2.4).

5.5.3 Anti-poisoning gate nel live stream

Il design offline del testing replica 1:1 il routing live, prevenendo fenomeni di *data leakage*. Tuttavia, l'ambiente di deployment espone alla minaccia di *data poisoning*. Un aggressore persistente potrebbe infiltrarsi immettendo attacchi lenti per abituare il grafo ai propri comportamenti nocivi, sfalsando lentamente lo scoring e la similarità del coseno. Il problema è mitigato dal gate anti-avvelenamento: sia l'internal state della **TGNMemory** che le code a scorrimento dei vicini temporali vengono aggiornate e *committate* in RAM solo nel caso in cui lo score della rete si posizioni sotto il limite calcolato per le minacce ($< threshold$). I pacchetti classificati come anomalia, inviati a OPA, vengono ignorati dal sistema di Intelligenza Artificiale, preservando incontaminato lo stato vettoriale del modello.

5.6 Quadro riepilogativo: confronto di tutti i modelli

La Tabella 5.9 consolida le prestazioni dei modelli in un unico quadro, organizzato in tre pannelli *per protocollo*: **sono validi solo i confronti all'interno dello stesso pannello**. Lo scopo è dare una lettura d'insieme della progressione — dalle baseline non relazionali al TGN, e dal modello a 4 nodi (v3) a quello con nodo *configuration* (v4) — senza dover ricomporre i numeri sparsi nelle Tabelle 5.1, 5.3, 5.4 e 5.5.

Modello / variante	AUC agg	AP agg	Lat. AUC	Lat. AP	Lat. Recall	agg Recall
<i>Pannello A — stream standard, 3 seed [42,7,123] (media, 200k ev.), soglia globale 1% FPR</i>						
TGN (v3, per-cookie)	0.965	0.922	0.913	—	0.166	0.665
GNN non temporale (ablation)	0.555	0.560	0.451	—	0.125 [†]	0.366
One-Class SVM	0.801	0.739	0.599	—	0.062	0.479
Isolation Forest	0.763	0.575	0.645	—	0.009	0.116
XGBoost (supervisionato) [‡]	0.974	0.948	0.929	—	0.339	0.772
<i>Pannello B — stream standard, 3 seed [42,7,123] (media), decisione cost-sensitive instradata</i>						
v3 (4 nodi, per-cookie)	0.965	0.922	0.913	0.522	0.350	0.758
v4 (nodo config, per-cookie)	0.964	0.897	0.930	0.575	0.577	0.829
v4 + dev:guest (deployato)*	0.965	0.926	0.922	0.652	0.532	0.811
<i>Pannello C — stream theft-rich, 3 seed (media), ablazioni controllate (save=False)</i>						
v4 (nodo config)	0.958	—	0.919	—	0.469	0.777
≈v3 (ablazione config)	0.956	—	0.913	—	0.334	0.744
v4 + dev:guest	0.961	—	0.911	0.632	0.488	0.787

Tabella 5.9: Quadro riepilogativo di tutti i modelli e varianti: **sono validi solo i confronti intra-pannello** (protocolli diversi). *Lat. Recall* e *agg Recall* sono a soglia globale 1% FPR nel Pannello A e a decisione instradata nei Pannelli B/C. [†] recall spurio (GNN, AUC≈caso); [‡] XGBoost *supervisionato* (upper-bound fuori categoria). * riga dev:guest (modello deployato) da un run appaiato successivo con codice aggiornato: i suoi valori *assoluti* non sono direttamente confrontabili con le righe v3/v4 soprastanti (prodotte prima di alcune migliorie); il confronto valido col per-cookie è il Δ *appaiato* di Tab. 5.6 (−0.030 lat AUC), non la differenza con questa tabella.

6 Validazione e Test delle API (Postman)

In questa sezione vengono presentati i risultati dei test di carico e affidabilità condotti sulle API dell'applicazione sviluppata. L'obiettivo dell'analisi è verificare la resilienza, i tempi di risposta e la stabilità del backend simulando diversi scenari di accesso concorrente, al fine di individuare potenziali colli di bottiglia o vulnerabilità legate alla disponibilità del servizio (es. mitigazione di attacchi di tipo *Denial of Service* elementari o problemi di gestione delle risorse).

6.1 Ambiente di Test e Premessa Metodologica

I test sono stati eseguiti all'interno di un ambiente controllato in locale (*localhost*). Per garantire la replicabilità dei risultati, si riportano di seguito le specifiche hardware delle macchine ospitanti su cui sono stati eseguiti sia l'applicazione sia l'istanza dello strumento di testing:

- **PC portatile:** CPU: AMD Ryzen 7 4000, Memoria RAM: 16 GB DDR4, GPU: Non dedicata
- **PC fisso:** CPU: AMD Ryzen 9 9900X, Memoria RAM: 32 GB DDR5, GPU: NVIDIA 5060
- **Strumento di Testing:** Postman

Nota di contesto: L'assenza di una GPU dedicata sul PC portatile influisce sulle prestazioni metriche dell'applicazione, in quanto il carico di lavoro del modello di IA è interamente gravato sulla CPU e sulla memoria RAM.

6.2 Analisi dei Risultati e Scenari di Carico

I test effettuati hanno simulato diverse curve di carico misurate in *Virtual Users* (UV). Di seguito vengono analizzati nel dettaglio i singoli scenari eseguiti.

6.2.1 PC portatile

Tabella 6.1: Metriche delle prestazioni registrate su Postman (PC portatile)

Scenario di Test	Profilo di Carico	Tempo Medio (Avg)	Tempo Max	Durata
Carico Costante	5 UV costanti	~950 ms	1.6 s	3 min
Rampa Lineare	Da 1 a 5 UV	~1.0 s (a regime)	–	Var.
Picco Repentino	Spike 5 UV	–	1.0 s	Impuls.
Picco Repentino	Spike 10 UV	–	2.0 s	Impuls.
Rampa Incrementale	Da 1 a 10 UV	~1.5 s (a regime)	–	Var.

6.2.2 PC fisso

Tabella 6.2: Metriche delle prestazioni registrate su Postman (PC fisso)

Scenario di Test	Profilo di Carico	Tempo Medio (Avg)	Tempo Max	Durata
Carico Costante	5 UV costanti	~21 ms	1 s	2 min
Rampa Lineare	Da 1 a 5 UV	~17 ms a regime	–	2 min
Carico Costante	20 UV costanti	~32 ms	1.6 s	2 min
Rampa Lineare	Da 1 a 20 UV	~16 ms a regime	–	Var.
Picco Repentino	Spike 20 UV	–	4.0 s	Impuls.
Rampa Lineare	Da 1 a 30 UV	~18 ms a regime	–	Var.
Rampa Lineare up&down	30 UV	18 ms (40 ms a regime)	1,65 s	–
Picco Repentino	Spike 30 UV	–	7.0 s	Impuls.
Rampa Lineare up&down	100 UV	268 ms (500 ms a regime)	3 s	2 min

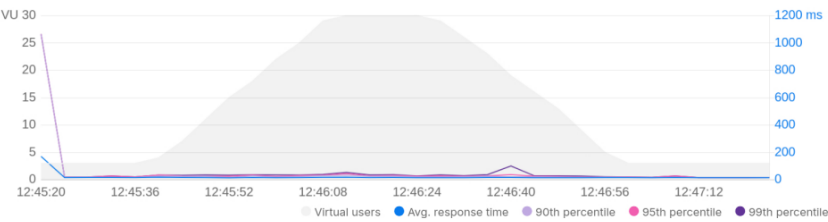


Figura 6.1: Grafico Peak 30 UV

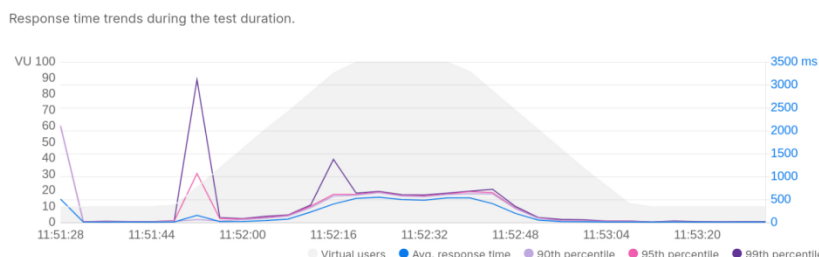


Figura 6.2: Grafico Peak 100 UV

Considerazioni Tecniche

L'esecuzione dei test di carico a volumi di traffico reali ha permesso di delineare con maggiore precisione i limiti di scalabilità e la robustezza del backend sviluppato. L'analisi dei dati aggregati nelle tabelle 6.1 e 6.2 evidenzia i seguenti comportamenti del sistema:

- **Efficienza in Condizioni Standard (Saturazione Bassa/Media):** Nello scenario a carico costante con 5 UV, il sistema risponde con un tempo medio estremamente ridotto (~ 21 ms). Questo valore, unito ai ~ 32 ms registrati con 20 UV costanti, dimostra l'ottima efficienza della logica di routing e della gestione dei thread.
- **Scalabilità Lineare e Curve di Ramp-up:** I test di *Ramp-up* lineare (fino a 5, 20 e 30 UV) mostrano un comportamento quasi ottimale, con tempi a regime stabili tra i 16 ms e i 18 ms. Il server si dimostra capace di allocare dinamicamente le risorse necessarie man mano che nuove connessioni vengono stabilite, senza subire degradazioni prestazionali premature.
- **Analisi degli Scenari di Stress Avanzati (30 e 100 UV):** Nel test *Ramp-up up&down* a 30 UV si nota un primo incremento del tempo a regime, che sale a 40 ms. La vera e propria soglia di stress strutturale viene tuttavia raggiunta nello scenario a 100 UV (*Ramp-up up&down*): in questo caso il tempo medio globale si attesta a 268 ms, ma subisce un raddoppio sensibile arrivando a 500 ms una volta raggiunto il pieno regime di carico.
- **Resistenza agli Spike:** Lo *Spike* a 20 UV mostra un tempo massimo di 4.0 s, che sale linearmente a 7.0 s nello *Spike* a 30 UV; anche nel *Peak* a 100 UV in fase di salita è presente un tempo massimo di risposta di 3.0 s. Questo incremento dei tempi di risposta massimi durante i picchi impulsivi evidenzia che il backend, pur non andando in *crash*, soffre di un fenomeno temporaneo di *saturazione delle risorse* quando riceve un numero anomalo di richieste improvvise, situazione che viene comunque assorbita superata la fase critica.